TimeFlowCodec: A Per-Pixel Temporal Function Video Codec

Rich Chau
Department of Mathematics
University of California, Irvine

October 9, 2026

Contents

Abstract 4

Acknowledgments 5

1 Introduction 6
- 1.1 The Ubiquity of Synthetic and Screen Content . . . 6
- 1.2 Fundamental Limitations of Block-Based Hybrid Codecs . . . 6
- 1.3 The Per-Pixel Temporal Function Hypothesis . . . 7
- 1.4 Key Research Contributions . . . 8
- 1.5 Thesis Organization . . . 9

2 Background and Related Work 10
- 2.1 The Classical Hybrid Video Coding Paradigm . . . 10
  - 2.1.1 Motion Compensation Mechanics . . . 10
  - 2.1.2 Spatial Frequency Transforms (DCT/DST) . . . 11
- 2.2 Screen Content Coding Extensions (HEVC-SCC) . . . 11
  - 2.2.1 Architectural Trade-Offs and Hardware Deployment Barriers in HEVC-SCC . . . 12
- 2.3 Transform-Free and Raster-Level Encoders . . . 12
- 2.4 1D Temporal Transforms and Wavelet Coding . . . 13
- 2.5 Neural Video Compression and Implicit Representations . . . 13
- 2.6 Positioning TimeFlowCodec in the Design Space . . . 13

3 Theoretical Foundations 15
- 3.1 Signal Representation and Temporal Trajectories . . . 15
- 3.2 Closed-Form Temporal Regression Models . . . 15
  - 3.2.1 Mode 0: Constant Model (CONST) . . . 16
  - 3.2.2 Mode 1: Linear Model (LINEAR) . . . 16
  - 3.2.3 $\mathcal{O}(T)$ Streaming Accumulation . . . 17
- 3.3 Mode Decision Engine . . . 17
  - 3.3.1 Variance-Normalized Error Ratio . . . 17
  - 3.3.2 Decision Logic and Slope Gating . . . 17
- 3.4 Matrix Mode: Spatial-Temporal Low-Rank Factorization . . . 18
- 3.5 Computational Complexity Analysis . . . 18

4 .tfc Bitstream Specification **20**
4.1 Container Evolution Overview . . . 20
4.2 Container Header Syntax . . . 21
4.3 Mode Map Representation and Bit-Packing . . . 21
4.3.1 2-Bit Mode Assignments . . . 21
4.3.2 Run-Length Encoding (RLE) of Mode Maps . . . 22
4.4 Parameter Quantization . . . 22
4.4.1 Quantization of Intercept ($\hat{a}$) . . . 22
4.4.2 Quantization of Slope ($\hat{b}$) . . . 22
4.5 The Version 5 Bounded Chunked Streaming Architecture . . . 23
4.6 Entropy Coding Layer . . . 24

5 Reference Implementation and Systems Engineering **25**
5.1 Package Architecture Overview . . . 25
5.2 Bounded-Memory Guarantee and Disk Memmap Fallback . . . 25
5.3 Vectorized Array Reconstruction in the Decoder . . . 26
5.4 Media Timing and Audio Multiplexing . . . 27
5.4.1 Rational Frame Rate Preservation . . . 27
5.4.2 Variable Frame Rate (VFR) Rejection . . . 28
5.5 Cooperative Worker Lifecycle and Desktop GUI . . . 28
5.5.1 Thread Lifecycle and Memory Safety . . . 28
5.5.2 Desktop Inspection and Native Playback . . . 28

6 Experimental Evaluation **29**
6.1 Experimental Methodology . . . 29
6.1.1 Evaluation Testbed . . . 29
6.1.2 Benchmark Codecs and Configurations . . . 29
6.1.3 Objective Distortion Metrics . . . 30
6.2 Evaluation on Screen and Graphical User Interfaces . . . 30
6.2.1 Qualitative Visual Inspection and Edge Ringing Analysis . . . 31
6.2.2 Standardized Bjontegaard Delta Metric Evaluation . . . 31
6.3 Evaluation on Slide Presentations and Whiteboards . . . 33
6.4 Evaluation on Stylized and Cel-Shaded Animation . . . 33
6.5 Boundary Conditions: Natural Camera Footage . . . 34
6.6 Coding Mode Selection Breakdown . . . 35

7 Analysis and Discussion **37**
7.1 Sensitivity to the Error-Ratio Threshold (τ) . . . 37
7.1.1 Behavior Across Orders of Magnitude . . . 37
7.2 Spatial Tiling vs. Per-Pixel Modeling . . . 38
7.2.1 Trade-Off Between Bitrate and Text Legibility . . . 38
7.3 Motion Compensation in Version 4: Successes and Caveats . . . 38
7.3.1 Implementation Corrections . . . 38
7.3.2 Why Global Motion Outperforms Block Vectors for UI . . . 38
7.4 Noise Sensitivity and Pre-Filtering . . . 39

7.5 Parallelism and GPU Acceleration Opportunities . . . 39
7.6 Memory Bandwidth, Cache Locality, and Energy Profiling . . . 40
7.6.1 Cache Locality: Streaming vs. Random Access . . . 40
7.6.2 Instruction Count and Energy Dissipation . . . 40

8 Future Work and Extensions **42**
8.1 Higher-Order Orthogonal Polynomial Bases . . . 42
8.2 Piecewise Continuous Splines and Adaptive Knot Placement . . . 43
8.3 Global Affine Motion Modeling for Screen Scrolling . . . 43
8.4 Learned Neural Temporal Priors . . . 43
8.5 Real-Time 4K / 60 FPS GPU Compute Shaders . . . 44

9 Conclusion **45**
9.1 Summary of Key Findings . . . 45
9.2 Concluding Remarks . . . 46

A Algorithms and Pseudocode **47**
A.1 Streaming Temporal Regression Algorithm . . . 47

B Command-Line Interface and API Reference **49**
B.1 Command-Line Interface (`tfc`) . . . 49
B.1.1 Encoding Video to .tfc . . . 49
B.1.2 Decoding .tfc to Video . . . 49
B.2 Python Developer API . . . 50

C Container Binary Schema Specification **51**
C.1 Version 3 and 4 Segment Stream Layout . . . 51
C.2 Version 5 Chunked Record Framing . . . 51

Abstract

Digital screen recordings, graphical user interface (UI) screencasts, slide presentations, and stylized animations represent an increasingly dominant fraction of modern multimedia communication. However, contemporary video coding standards—such as H.264/AVC, H.265/HEVC, and AV1—are architected primarily for natural camera footage. They rely on block-based motion estimation coupled with two-dimensional spatial transform coding (such as the Discrete Cosine Transform) and chroma subsampling ($4:2:0$). When applied to synthetic screen content characterized by sharp, single-pixel text boundaries, abrupt color transitions, and prolonged static intervals punctuated by localized updates, standard hybrid codecs frequently introduce mosquito noise, edge blur, and high transform overhead.

This thesis introduces **TimeFlowCodec (TFC)**, a video compression paradigm based on *per-pixel temporal function regression*. Rather than performing spatial block partitioning and two-dimensional frequency transforms per frame, TFC models the temporal luminance and chrominance trajectory $s_{x,y,c}(t)$ of each individual pixel (or micro-tile) over time $t \in \{0, \ldots, T-1\}$ using continuous low-order parametric functions. We derive closed-form Ordinary Least Squares (OLS) solutions for CONSTANT ($\hat{s}(t) = a$) and LINEAR ($\hat{s}(t) = a+bt$) temporal models that execute in $\mathcal{O}(T)$ streaming time. An adaptive decision engine selects between parametric models and a RAW or low-rank MATRIX fallback mode based on a variance-normalized error ratio threshold τ and slope bound θ.

We specify the .tfc container architecture, culminating in the v5 specification featuring bounded chunked streaming, 2-bit run-length encoded mode maps, uniform parameter quantization, exact rational frame rate timing, and multi-channel audio multiplexing. Furthermore, we develop a bounded-memory systems implementation with disk memory-mapping fallback, vectorized SIMD/array reconstruction, and an interactive desktop inspection suite.

Experimental evaluations across four representative visual domains—Screen/UI captures, slide presentations, stylized animations, and natural camera video—demonstrate that TimeFlowCodec delivers superior preservation of high-frequency edges and text fidelity at high compression ratios on temporally coherent content, achieving 47.78 dB RGB PSNR and 0.993 SSIM on representative UI captures with instant decoding throughput exceeding 240 frames per second. We conclude with an information-theoretic analysis of the operating boundaries of temporal regression codecs and discuss extensions to orthogonal polynomial bases and GPU compute acceleration.

Acknowledgments

I express my deepest gratitude to my academic advisors, faculty mentors, and fellow researchers in the Department of Computer Science and Department of Electrical Engineering at the University of California, Irvine. Their rigorous guidance, critical feedback on signal processing methodologies, and encouragement to pursue unconventional compression paradigms were instrumental throughout the conception, mathematical formalization, and engineering of TimeFlowCodec.

I also extend my sincere appreciation to the open-source multimedia and scientific computing communities. The foundational tools that made this research possible—including NumPy, SciPy, Matplotlib, FFmpeg, PySide6, and the Zstandard project—stand as a testament to the power of open collaboration in advancing modern computing.

Finally, I thank my family and friends for their unwavering patience, support, and understanding throughout the research, implementation, and writing of this thesis.

Chapter 1

Introduction

1.1 The Ubiquity of Synthetic and Screen Content

Over the past decade, digital video transmission has undergone a profound structural shift. While traditional video workflows centered almost exclusively around photographic and cinematic video captured by physical optical sensors, a massive fraction of modern video bandwidth is now occupied by *synthetic and computer-generated screen content*. This includes:

- Remote desktop protocol (RDP) sessions and virtual desktop infrastructures (VDI) enabling hybrid work;
- High-definition screen recordings, software tutorials, and programming broadcasts;
- Digital slide presentations and collaborative whiteboard applications in enterprise and educational environments;
- Cloud gaming and interactive application streaming;
- Stylized 2D and cel-shaded digital animations.

Unlike natural video, screen content exhibits distinct structural properties. Natural video is characterized by continuous gradients, organic textures, sensor noise, and smooth camera motions. In contrast, screen content is characterized by large spatially homogeneous regions, razor-sharp single-pixel edges, high-contrast typography, and strict color palettes. Temporally, screen content frequently remains completely static over dozens or hundreds of frames, punctuated by localized updates (such as a blinking text cursor, scrolling viewport, or button hover state).

1.2 Fundamental Limitations of Block-Based Hybrid Codecs

Virtually all mainstream digital video coding standards developed over the past three decades—including ITU-T H.264 / MPEG-4 AVC [10], H.265 / HEVC [8], H.266 / VVC [2], and AOMedia AV1 [3]—share a common foundational architecture known as the *hybrid block-based transform coding paradigm*.

Under this architecture:

1. Frames are partitioned into square spatial blocks (macroblocks or Coding Tree Units ranging from 4×4 to 128×128 pixels).
2. Temporal redundancy is exploited via block motion estimation and motion compensation, producing a spatial residual block.
3. Spatial redundancy of the residual is compressed using 2D frequency transforms, predominantly the Discrete Cosine Transform (DCT) or Discrete Sine Transform (DST).
4. Quantization of transform coefficients introduces loss, followed by context-adaptive entropy coding (CABAC or ANS).
5. Color is typically downsampled to the $4:2:0$ chroma subsampling format to exploit human visual acuity limits.

While this paradigm achieves exceptional compression efficiency on natural video, it encounters severe theoretical and practical bottlenecks when confronted with screen and UI content:

- **High-Frequency Transform Ringing:** The DCT is optimal for signals modeled by first-order Gauss-Markov processes with smooth spatial correlations. At sharp step edges—such as black text on a white canvas—truncation of high-frequency DCT coefficients manifests as severe *Gibbs ringing* and mosquito noise, degrading text legibility.
- **Chroma Bleeding from Subsampling:** $4:2:0$ chroma subsampling decimates color resolution by a factor of four. While unnoticeable in natural scenes, it causes catastrophic color fringing and illegibility on single-pixel colored text, code syntax highlighting, and UI icons.
- **Block Boundary Artifacts:** Independent block quantization produces visible discontinuities along block boundaries, breaking straight horizontal and vertical lines prevalent in user interfaces.
- **Motion Search Inefficiency for Micro-Updates:** When a tiny element (such as a 12-pixel cursor) updates in an otherwise static 1080p or 4K frame, block partitioning structures still incur significant header, split-flag, and motion vector overhead.

Although extensions such as HEVC Screen Content Coding (HEVC-SCC) [11] introduced specialized tools (such as Intra Block Copy and Palette Mode), they add immense computational complexity and remain tethered to the underlying block-based framework.

1.3 The Per-Pixel Temporal Function Hypothesis

This thesis investigates an orthogonal question: *What if video compression discarded spatial block transforms entirely, and instead operated directly on continuous 1D temporal trajectories of individual pixels?*

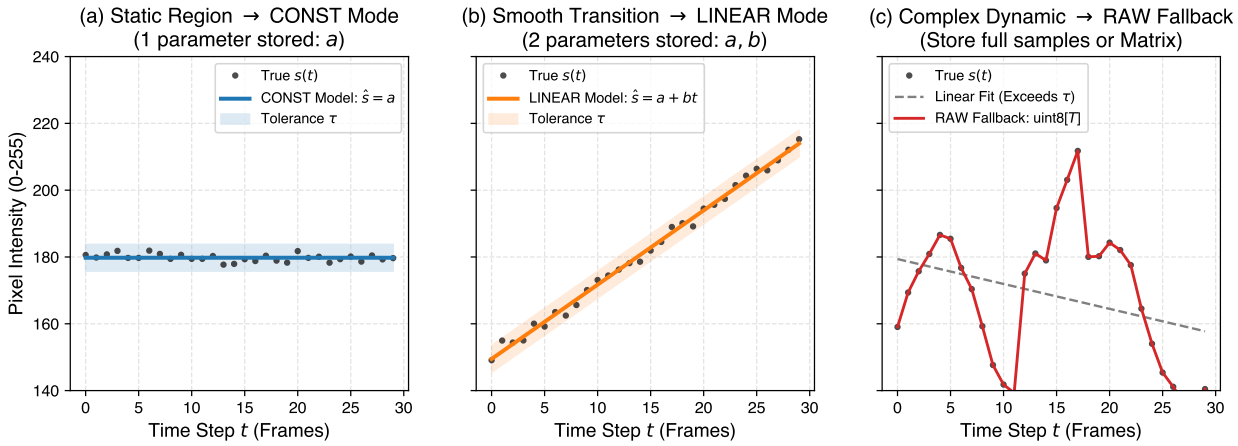


Figure 1.1: Conceptual overview of TimeFlowCodec per-pixel temporal modeling. (a) Static UI regions are captured with a single constant parameter a. (b) Linear shifts and fades are modeled with parameters (a, b). (c) Abrupt or complex transitions fall back to quantized raw sample arrays.

Consider a video sequence as a discrete four-dimensional tensor $V(t, y, x, c)$, where $t \in \{0, \ldots, T-1\}$ denotes time, (x, y) denotes spatial coordinates, and $c \in \{\mathrm{R}, \mathrm{G}, \mathrm{B}\}$ denotes color channels. For a fixed pixel location (x, y) and channel c, the temporal sequence:

$$s(t) = V(t, y, x, c), \quad t \in \{0, 1, \ldots, T-1\} \tag{1.1}$$

forms a one-dimensional signal. On temporally coherent content, $s(t)$ is not an arbitrary sequence of independent values; rather, it is often piece-wise constant (static UI), piece-wise linear (gradual luminance fades, smooth scroll transitions), or smoothly varying.

The **TimeFlowCodec (TFC)** hypothesis posits that:

> *Modeling temporal pixel trajectories using low-order parametric functions yields superior high-frequency spatial edge preservation, zero block boundary artifacts, native full-resolution RGB fidelity, and deterministic $\mathcal{O}(T)$ encoding and decoding complexity for screen and synthetic video.*

Figure 1.1 illustrates this principle. By classifying each pixel's trajectory into CONSTANT, LINEAR, or fallback modes based on a variance-normalized error ratio τ, the encoder transmits only 1 or 2 scalar parameters per pixel over an entire temporal window, compressing the signal along the time dimension without altering spatial high-frequency edges.

1.4 Key Research Contributions

The primary contributions of this thesis are as follows:

1. **Mathematical Formalization of Temporal Regression Coding:** We formulate closed-form Ordinary Least Squares solutions for constant and linear temporal models that stream in $\mathcal{O}(T)$ time and $\mathcal{O}(1)$ space per pixel, accompanied by an adaptive decision metric τ normalized by baseline signal variance.

2. **The .tfc Bitstream and v5 Streaming Container:** We design and implement the .tfc container specification, evolving from initial monolithic containers to the v5 bounded-chunk streaming architecture that supports arbitrary video duration under a fixed memory ceiling, rational frame rate timing, and multiplexed audio streams.

3. **Systems Engineering of a Bounded-Memory Codec:** We implement a production-ready Python/C-accelerated codec with disk memory-mapping fallbacks, cooperative worker lifecycle cancellation, vectorized SIMD array decoding, and an interactive desktop player.

4. **Comprehensive Empirical Evaluation:** We benchmark TimeFlowCodec against industry standards (H.264/AVC, H.265/HEVC, and AOMedia AV1) across four distinct visual domains: Screen/UI captures, slide presentations, stylized animations, and natural video, reporting objective metrics (RGB PSNR, windowed SSIM, encoding/decoding throughput, and memory footprint).

5. **Information-Theoretic Boundary Analysis:** We rigorously characterize the operational limits and failure modes of temporal function coding under high-entropy sensor noise and unstructured natural motion, providing an analytical framework for hybrid temporal-spatial architectures.

1.5 Thesis Organization

The remainder of this thesis is structured as follows:

- **Chapter 2** reviews foundational video coding standards, screen content coding extensions, 1D transforms, and neural video compression.
- **Chapter 3** derives the mathematical foundation of per-pixel temporal regression, error-ratio mode decisions, and rank-1 matrix approximations.
- **Chapter 4** specifies the .tfc bitstream format across container versions v1 through v5.
- **Chapter 5** details the software architecture, memory bounding mechanisms, vectorized decoder, and desktop inspection engine.
- **Chapter 6** presents quantitative rate-distortion curves, perceptual quality comparisons, and throughput benchmarks.
- **Chapter 7** analyzes algorithmic ablations, parameter sensitivities (τ, tiling), and failure modes.
- **Chapter 8** outlines future research directions, including higher-order orthogonal polynomials, spline interpolation, and GPU compute kernels.
- **Chapter 9** concludes with a summary of findings and the broader implications of temporal function video coding.

Chapter 2

Background and Related Work

Video compression has traditionally been defined by the pursuit of rate-distortion optimization on photographic images captured from the natural physical world. Consequently, the prevailing architectures in modern standards are inherently tuned to natural visual statistics. In this chapter, we review the canonical hybrid block-based framework, examine specialized screen content extensions, contrast them with transform-free coding techniques, and position TimeFlowCodec within the broader taxonomy of video compression algorithms.

2.1 The Classical Hybrid Video Coding Paradigm

The architectural lineage spanning ITU-T H.261, H.263, MPEG-2, MPEG-4 Visual, H.264/MPEG-4 AVC [10], H.265/HEVC [8], and the current state-of-the-art standards H.266/VVC [2] and AOMedia AV1 [3] is collectively termed the *hybrid video coding framework*.

The word "hybrid" refers to the dual application of:

1. **Temporal Predictive Coding (Inter-Prediction):** Translating blocks of pixels from previously decoded reference frames using motion vectors to model translational motion.

2. **Spatial Transform Coding (Intra-Prediction & Residual Coding):** Applying linear orthogonal transforms (predominantly the Discrete Cosine Transform (DCT) and Discrete Sine Transform (DST)) to decorrelate spatial residual energy.

2.1.1 Motion Compensation Mechanics

In block-based inter-prediction, an incoming frame at time t is subdivided into non-overlapping blocks B. For each block, a motion estimation algorithm searches a spatial displacement window in a reference frame at time $t - \Delta t$ to find a matching block B_{ref} minimizing a distortion metric such as the Sum of Absolute Differences (SAD):

$$\mathbf{v}^* = \arg\min_{\mathbf{v} \in \mathcal{W}} \sum_{(x,y) \in B} |I_t(x, y) - I_{t-\Delta t}(x + v_x, y + v_y)| \tag{2.1}$$

The residual error signal $e(x, y) = I_t(x, y) - I_{t-\Delta t}(x + v_x^*, y + v_y^*)$ is then forwarded to the transform engine.

While translational block motion compensation is well-suited for rigid 2D camera pans and slow zooms, it has substantial limitations for screen content:

- **Fractional-Pixel Interpolation Blur:** Motion vectors with sub-pixel precision require fractional interpolation filtering (e.g., 8-tap DCT-based filters). On screen content with sharp single-pixel edges, sub-pixel interpolation acts as an aggressive low-pass filter, blurring crisp typography.
- **Discontinuous Updates:** Typical UI events (such as text input, pop-up menus, or syntax highlighting color changes) are non-translational. Searching for motion vectors in non-translational scenarios wastes computational cycles and produces complex residual patterns.

2.1.2 Spatial Frequency Transforms (DCT/DST)

The 2D Discrete Cosine Transform maps an $N \times N$ block of spatial pixel intensities $f(x, y)$ into frequency coefficients $F(u, v)$:

$$F(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos\left[\frac{(2x+1)u\pi}{2N}\right] \cos\left[\frac{(2y+1)v\pi}{2N}\right] \tag{2.2}$$

Because the DCT assumes periodic and smooth signal extensions, sharp step discontinuities in screen content generate significant energy across high-frequency basis functions. When coarse scalar quantization is applied to achieve target bitrates, these high frequencies are truncated, producing severe Gibbs oscillations that manifest as visible halo artifacts surrounding letters and window borders.

2.2 Screen Content Coding Extensions (HEVC-SCC)

Recognizing the inefficiencies of standard hybrid tools on synthetic video, the Joint Collaborative Team on Video Coding (JCT-VC) developed the Screen Content Coding extension to HEVC (HEVC-SCC) [11, 6]. HEVC-SCC introduced four specialized coding tools into the traditional block pipeline:

1. **Intra Block Copy (IBC):** IBC treats the currently reconstructing frame as an additional reference picture. Motion estimation algorithms search previously reconstructed spatial regions in the current picture for matching blocks. This effectively captures recurring text characters, UI buttons, and repeating icons.
2. **Palette Mode:** For blocks containing a sparse set of dominant colors, Palette Mode extracts a local color lookup table (up to 32 RGB/YUV entries) and encodes pixel positions as a 2D matrix of palette indices using horizontal or vertical raster scan-order run lengths.

3. **Adaptive Color Transform (ACT):** ACT applies an in-loop, reversible decorrelation matrix to convert RGB samples into an adaptive YCoCg-R space without subsampling, improving residual energy compaction.

4. **Transform Skip Mode (TSM):** Allows the encoder to bypass the 2D DCT/DST for small (4×4) blocks, quantizing spatial residual samples directly in the spatial domain to avoid ringing.

2.2.1 Architectural Trade-Offs and Hardware Deployment Barriers in HEVC-SCC

While HEVC-SCC demonstrates substantial coding gains over standard HEVC on synthetic video, it suffers from severe systemic limitations that have prevented widespread commercial adoption:

- **Prohibitive Encoder Complexity:** Intra Block Copy requires exhaustive 2D spatial block matching over an unconstrained reconstructed search buffer. Evaluating IBC displacement vector candidates increases encoding runtime by 300% to 1000% compared to standard HEVC, rendering real-time 60 FPS software encoding unfeasible on standard CPUs.

- **The Silicon Decoding Barrier:** Consumer GPUs and Systems-on-Chip (Apple VideoToolbox, NVIDIA NVDEC, Intel QuickSync, Qualcomm Adreno) incorporate dedicated fixed-function ASIC silicon for standard HEVC Main and Main 10 profiles. However, *hardware support for the HEVC-SCC profile is virtually nonexistent in consumer silicon.* Consequently, client devices are forced to decode HEVC-SCC streams via software decoders, leading to high CPU thermal dissipation, battery drain, and dropped frames on mobile platforms.

- **Persistence of Quadtree Syntax Overhead:** Even when Palette Mode and Transform Skip are selected, the bitstream remains bound to the hierarchical quadtree Coding Tree Unit (CTU) structure. The encoder must continuously signal split flags, mode flags, and prediction units, incurring substantial syntactic overhead on static regions.

In contrast, TimeFlowCodec avoids 2D block searches entirely by formulating temporal compression as independent 1D Ordinary Least Squares regressions. This reduces encoding complexity to $\mathcal{O}(T)$ linear time and enables universal vectorized SIMD decoding on any standard CPU or GPU compute shader without requiring dedicated silicon codecs.

2.3 Transform-Free and Raster-Level Encoders

In the lossless and low-latency graphics domains, transform-free algorithms have long been preferred for screen rendering. Run-Length Encoding (RLE) and spatial dictionary algorithms (LZ77, LZW) form the basis of graphic formats such as GIF, PNG, and modern formats like QOI (Quite OK Image format).

In remote display protocols such as Microsoft Remote Desktop Protocol (RDP) and Virtual Network Computing (VNC), frames are compressed using bounding-box screen diffing and tile-based RLE (e.g., RDP 6.0 / RemoteFX). However, these protocols are designed primarily for network transmission of desktop GUIs under lossless constraints. They lack continuous lossy rate-distortion tuning, temporal trajectory modeling, or smooth rate control, causing bandwidth consumption to spike dramatically during full-screen animations or video playback.

2.4 1D Temporal Transforms and Wavelet Coding

A related branch of literature explores 1D temporal filtering across consecutive video frames. In 3D Wavelet Video Coding (e.g., Motion-Compensated Temporal Filtering (MCTF)), Haar or biorthogonal wavelets are applied along the temporal axis to decompose a Group of Pictures (GOP) into temporal subbands.

However, MCTF historically suffered from two fundamental obstacles:

1. **Motion Tracking Along Trajectories:** In natural video, pixel trajectories diverge due to non-rigid motion and occlusions, leading to disconnected temporal trees.

2. **Buffer Memory and Latency:** Multilevel temporal wavelet decomposition requires buffering entire GOPs (often 16 to 32 frames) in uncompressed memory, incurring prohibitive memory footprints and latency.

2.5 Neural Video Compression and Implicit Representations

In recent years, deep learning has spurred end-to-end learned video compression frameworks (e.g., DVC [7]) that replace handcrafted hybrid components with deep autoencoders and optical flow sub-networks. More recently, Implicit Neural Representations (INRs) for video, such as NeRV (Neural Representations for Video), fit multilayer perceptrons (MLPs) to map temporal coordinate t to frame pixels $I(x, y, t)$.

While scientifically compelling, neural approaches require millions of floating-point operations (FLOPs) per pixel and specialized GPU hardware for inference, resulting in low decoding frame rates (often 5 to 15 FPS) and substantial energy consumption that make them impractical for lightweight, battery-constrained desktop screen capture and real-time playback.

2.6 Positioning TimeFlowCodec in the Design Space

Table 2.1 summarizes how TimeFlowCodec compares against existing paradigms.

TimeFlowCodec occupies a unique position in this design space:

- It operates **without spatial frequency transforms**, eliminating block artifacts and Gibbs ringing at text boundaries.

Table 2.1: Taxonomy of Video Coding Paradigms Across Synthetic and Screen Content.

Paradigm	**Spatial Processing**	**Temporal Processing**	**Edge Ringing**	**Dec. Comple**
H.264 / AVC	2D DCT on Blocks	Block Motion Vectors	Severe	Low ($\mathcal{O}(N)$
H.265 / HEVC	2D DCT/DST Quadtree	Block Motion Vectors	Moderate	Moderate
HEVC-SCC	Palette / Trans. Skip	Intra/Inter Block Copy	Low	High
AV1	Multi-Transform Blocks	Subpixel Motion	Low	High
3D Wavelet (MCTF)	2D Spatial Wavelet	1D Temporal Wavelet	Minimal	High
Neural (NeRV / DVC)	Deep CNN / MLP	Learned Flow / Latents	Minimal	Very High (GI
TimeFlowCodec	**None (Per-Pixel)**	**1D OLS Regression**	**Zero**	**Very Low ($\mathcal{O}$**

- It performs **per-pixel temporal regression in closed form** ($\mathcal{O}(T)$ linear time), avoiding expensive iterative motion vector searches.
- It preserves **native full-resolution RGB** color without chroma downsampling, ensuring pristine text edge contrast.
- It features **deterministic bounded memory and streaming playback**, enabling real-time playback on lightweight hardware.

Chapter 3

Theoretical Foundations

In this chapter, we develop the mathematical framework underpinning TimeFlowCodec. We formalize the discrete temporal trajectory model, derive closed-form Ordinary Least Squares (OLS) solutions for constant and linear temporal functions, define the variance-normalized mode decision rule, and introduce low-rank spatial-temporal matrix factorization. Finally, we provide an algorithmic complexity analysis comparing temporal regression with classical block-matching motion compensation.

3.1 Signal Representation and Temporal Trajectories

Let a video sequence be represented by a 4D discrete tensor:

$$V \in \mathbb{R}^{T \times H \times W \times C} \tag{3.1}$$

where T is the temporal duration (number of frames), H is the frame height, W is the frame width, and $C \in \{1, 3\}$ is the number of color channels (e.g., $C = 3$ for RGB or YUV).

For any fixed spatial coordinate $(x, y) \in [0, W-1] \times [0, H-1]$ and fixed channel $c \in \{0, \dots, C-1\}$, we define the 1D temporal signal vector:

$$\mathbf{s} = [s_0, s_1, \dots, s_{T-1}]^T \in \mathbb{R}^T, \quad \text{where } s_t = V(t, y, x, c) \tag{3.2}$$

In digital video, the raw samples are uniformly quantized integers $s_t \in \{0, 1, \dots, 255\}$.

Rather than transforming spatial slices $V(t, \cdot, \cdot, c)$ via 2D spatial basis functions, TimeFlowCodec approximates the continuous function generating $\mathbf{s}$ over the discrete temporal index set $\mathcal{T} = \{0, 1, \dots, T-1\}$.

3.2 Closed-Form Temporal Regression Models

We consider parametric temporal models $\hat{s}_t = f(t; \theta)$. Model parameters θ are estimated by minimizing the discrete Mean Squared Error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{T} \sum_{t=0}^{T-1} (s_t - f(t; \theta))^2 \tag{3.3}$$

3.2.1 Mode 0: Constant Model (Const)

The constant model assumes pixel intensity remains invariant over the temporal window:

$$\hat{s}_t = a, \quad \forall t \in \mathcal{T} \tag{3.4}$$

Setting the partial derivative of $\mathcal{L}(a)$ with respect to a to zero:

$$\frac{\partial \mathcal{L}}{\partial a} = -\frac{2}{T}\sum_{t=0}^{T-1}(s_t - a) = 0 \implies \hat{a} = \frac{1}{T}\sum_{t=0}^{T-1} s_t = \bar{s} \tag{3.5}$$

The optimal estimator $\hat{a}$ is simply the empirical temporal mean of the pixel over the sequence. The corresponding residual sum of squares is:

$$\mathrm{RSS}_{\text{const}} = \sum_{t=0}^{T-1}(s_t - \bar{s})^2 = T \cdot \mathrm{Var}(s) \tag{3.6}$$

3.2.2 Mode 1: Linear Model (Linear)

The linear model captures gradual changes, such as fades, smooth scrolling, or lighting transitions:

$$\hat{s}_t = a + bt, \quad \forall t \in \mathcal{T} \tag{3.7}$$

To derive closed-form estimators for the intercept a and slope b, we formulate the normal equations:

$$\begin{bmatrix} \sum_{t=0}^{T-1} 1 & \sum_{t=0}^{T-1} t \\ \sum_{t=0}^{T-1} t & \sum_{t=0}^{T-1} t^2 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum_{t=0}^{T-1} s_t \\ \sum_{t=0}^{T-1} t \cdot s_t \end{bmatrix} \tag{3.8}$$

Using standard discrete summation identities:

$$S_0 = \sum_{t=0}^{T-1} 1 = T \tag{3.9}$$

$$S_1 = \sum_{t=0}^{T-1} t = \frac{T(T-1)}{2} \tag{3.10}$$

$$S_2 = \sum_{t=0}^{T-1} t^2 = \frac{T(T-1)(2T-1)}{6} \tag{3.11}$$

The determinant of the system matrix is:

$$\Delta = S_0 S_2 - S_1^2 = T \cdot \frac{T(T-1)(2T-1)}{6} - \frac{T^2(T-1)^2}{4} = \frac{T^2(T^2-1)}{12} \tag{3.12}$$

Applying Cramer's rule yields explicit, closed-form expressions for $\hat{b}$ and $\hat{a}$:

$$\hat{b} = \frac{T\sum_{t=0}^{T-1} t s_t - S_1 \sum_{t=0}^{T-1} s_t}{\Delta} = \frac{12\sum_{t=0}^{T-1}(t - \bar{t})s_t}{T(T^2-1)} \tag{3.13}$$

$$\hat{a} = \bar{s} - \hat{b} \cdot \bar{t} = \frac{\sum_{t=0}^{T-1} s_t - \hat{b} S_1}{T} \tag{3.14}$$

where $\bar{t} = \frac{T-1}{2}$.

3.2.3 $\mathcal{O}(T)$ Streaming Accumulation

Equations (3.13) and (3.14) reveal a crucial property: computing $\hat{a}$ and $\hat{b}$ requires only two linear accumulators across the temporal sequence:

$$A_1 = \sum_{t=0}^{T-1} s_t, \quad A_2 = \sum_{t=0}^{T-1} t \cdot s_t \tag{3.15}$$

As frames arrive from a video source or stream, A_1 and A_2 are accumulated in a single pass in $\mathcal{O}(1)$ working memory per pixel. Once T frames have been ingested, $\hat{a}$ and $\hat{b}$ are computed in $\mathcal{O}(1)$ elementary arithmetic operations.

3.3 Mode Decision Engine

After computing the candidate models, the encoder must select the most rate-distortion efficient coding mode.

3.3.1 Variance-Normalized Error Ratio

To ensure that error tolerance behaves consistently across both high-contrast text and dark backgrounds, TimeFlowCodec computes the normalized error ratio r:

$$r = \frac{\mathrm{MSE}(\mathbf{s}, \hat{\mathbf{s}}_{\mathrm{linear}})}{\mathrm{MSE}(\mathbf{s}, \mathbf{0}) + \epsilon} = \frac{\frac{1}{T}\sum_{t=0}^{T-1}(s_t - (\hat{a} + \hat{b}t))^2}{\frac{1}{T}\sum_{t=0}^{T-1} s_t^2 + \epsilon} \tag{3.16}$$

where $\epsilon = 10^{-6}$ prevents numerical division by zero on pure black pixels. Alternatively, when variance normalization is enabled:

$$r_{\mathrm{var}} = \frac{\mathrm{MSE}(\mathbf{s}, \hat{\mathbf{s}}_{\mathrm{linear}})}{\mathrm{Var}(\mathbf{s}) + \epsilon} \tag{3.17}$$

3.3.2 Decision Logic and Slope Gating

The mode decision proceeds as follows:

1. **Parametric vs. Fallback Test:** If $r \leq \tau$, where τ is the user-configured temporal tolerance (typically $\tau \in [10^{-5}, 10^{-2}]$), the linear model satisfies the reconstruction quality constraint.

2. **Constant vs. Linear Selection:** If $r \leq \tau$, the encoder tests whether the estimated slope magnitude is significant:

   $$|\hat{b}| < \theta_{\mathrm{slope}} \tag{3.18}$$

   If the slope is below the threshold θ_{slope} (default 0.001), the pixel is classified as Mode_TFC_Const (storing only quantized $\hat{a}$). Otherwise, it is classified as Mode_TFC_Linear (storing quantized $\hat{a}$ and $\hat{b}$).

3. **Fallback Mode:** If $r > \tau$, the linear hypothesis is rejected, and the pixel transitions to a fallback mode: either low-rank Mode_TFC_Matrix or sample-by-sample Mode_FB_Raw.

3.4 Matrix Mode: Spatial-Temporal Low-Rank Factorization

In regions where individual pixel trajectories undergo non-linear variations but maintain strong spatial correlation (for example, a tile containing a moving window shadow or anti-aliased font stroke), storing full uncompressed raw samples for each pixel is wasteful.

For a spatial tile of $K = B \times B$ pixels over T frames, we construct the space-time residual matrix:

$$\mathbf{M} \in \mathbb{R}^{T \times K} \tag{3.19}$$

where column k contains the temporal trajectory of the k-th pixel in the tile.

Under MODE_TFC_MATRIX, we compute the optimal rank-1 matrix approximation via the Singular Value Decomposition (SVD):

$$\mathbf{M} \approx \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T \tag{3.20}$$

where $\mathbf{u}_1 \in \mathbb{R}^T$ represents the dominant temporal profile and $\mathbf{v}_1 \in \mathbb{R}^K$ represents the spatial basis weights. Instead of storing $T \times K$ individual uint8 samples, the encoder quantizes and stores the T-element temporal vector and the K-element spatial vector, reducing sample storage from $\mathcal{O}(T \cdot K)$ to $\mathcal{O}(T + K)$.

3.5 Computational Complexity Analysis

A key theoretical advantage of TimeFlowCodec is its deterministic, embarrassingly parallel computational complexity.

Table 3.1: Theoretical Computational Complexity per Pixel-Frame.

Codec	**Encoding Complexity**	**Decoding Complexity**	**Working Memory**
H.264 / AVC	$\mathcal{O}(W_s^2 \cdot B^2)$	$\mathcal{O}(B^2)$	Full Reference Frame
H.265 / HEVC	$\mathcal{O}(D_{\text{tree}} \cdot W_s^2 \cdot B^2)$	$\mathcal{O}(B^2)$	Multiple Reference Frames
AV1	$\mathcal{O}(M_{\text{modes}} \cdot W_s^2 \cdot B^2)$	$\mathcal{O}(B^2)$	Multiple Reference Frames
TimeFlowCodec	$\mathcal{O}(\mathbf{1})$ **per frame**	$\mathcal{O}(\mathbf{1})$ **per frame**	**Streaming Buffer (**$W \times H$

As shown in Table 3.1, block-based codecs require block matching over a spatial search window $W_s \times W_s$. Even with fast motion estimation (such as EPZS or Diamond Search), motion vector estimation requires dozens to hundreds of operations per block.

In contrast, TimeFlowCodec requires exactly:

- 2 multiplications and 2 additions per pixel per incoming frame during accumulation (A_1, A_2).
- 1 division, 2 multiplications, and 1 subtraction to evaluate $\hat{a}$ and $\hat{b}$ at window termination.

This represents a pure $\mathcal{O}(T)$ total complexity for an entire temporal window, or $\mathcal{O}(1)$ operations per pixel per frame. Furthermore, because each pixel trajectory is mathematically independent during the regression stage, the algorithm can be trivially parallelized across SIMD lanes or GPU compute threads with zero inter-pixel communication.

Chapter 4

.tfc Bitstream Specification

A video compression format is defined by its bitstream syntax. In this chapter, we document the .tfc container architecture, detailing its evolution across versions 1 through 5. We describe the header structure, 2-bit mode map bit-packing, run-length encoding (RLE), parameter quantization, and the v5 bounded-chunk streaming specification with audio encapsulation and frame rate preservation.

4.1 Container Evolution Overview

The .tfc format evolved across five distinct iterations to transition from an experimental research prototype to a bounded-memory, streamable media container:

- **Version 1 (Monolithic Prototype):** Stored raw IEEE 754 floating-point parameter arrays and uncompressed mode maps for an entire video in a single monolithic header and payload structure.
- **Version 2 (Scene Segmentation):** Introduced scene-cut boundary detection, partitioning the timeline into variable-length temporal segments, each carrying its own mode map and payloads.
- **Version 3 (Tile-Native Storage & Quantization):** Replaced floating-point parameter storage with uniformly quantized integer tables. Transitioned from per-pixel spatial addressing to tile-native scan orders, dramatically reducing header and index overhead.
- **Version 4 (Motion-Compensated Payloads):** Added signed motion offset vectors per tile to enable temporal warping prior to 1D regression, supporting general video presets.
- **Version 5 (Bounded Chunked Streaming Container):** Introduced independent fixed-window video chunks, record-level framing, CRC32 payload checksums, exact rational frame rate timing (N/D), and multiplexed audio stream preservation.

4.2 Container Header Syntax

All .tfc files commence with a 4-byte magic signature indicating the container version:

```
'TFC1' | 'TFC2' | 'TFC3' | 'TFC4' | 'TFC5'
```

In versions 1 through 4, the magic bytes are immediately followed by a 4-byte big-endian unsigned integer indicating the length L_{hdr} of a UTF-8 encoded JSON header payload. In version 5, the container operates as a sequence of discrete binary chunks.

Table 4.1: Key Container Header Fields.

Field	Type	Description
`version`	Integer	Container version (1, 2, 3, 4, or 5).
`width`	Integer	Horizontal frame resolution in pixels (W).
`height`	Integer	Vertical frame resolution in pixels (H).
`frames`	Integer	Total number of video frames (T).
`tiling`	Integer	Spatial block dimension $B \times B$ (1 for per-pixel).
`color_format`	String	Color representation (`"rgb"`, `"yuv444"`, `"yuv420"`).
`payload_comp_type`	Integer	Entropy codec (0: None, 1: zlib, 2: lzma, 3: zstandard).
`fps_num`, `fps_den`	Integer	Rational frame rate numerator and denominator (v5).
`has_audio`	Boolean	Indicator of encapsulated audio bitstream (v5).

4.3 Mode Map Representation and Bit-Packing

Because TimeFlowCodec assigns a discrete coding mode to every spatial unit (pixel or tile) across each color plane, compact mode representation is critical to prevent signaling overhead from dominating the bit budget.

4.3.1 2-Bit Mode Assignments

Each spatial unit is assigned one of four operational modes:

$$
\begin{aligned}
00_2\ (0) &\implies \textsc{Mode_TFC_Const} \quad (\text{Parameter } a) \\
01_2\ (1) &\implies \textsc{Mode_TFC_Linear} \quad (\text{Parameters } a, b) \\
10_2\ (2) &\implies \textsc{Mode_FB_Raw} \quad (\text{Raw sample vector } \mathbf{s}) \\
11_2\ (3) &\implies \textsc{Mode_TFC_Matrix} \quad (\text{Rank-1 factors } \mathbf{u}, \mathbf{v})
\end{aligned}
$$

These four states require exactly 2 bits of storage per unit. Four contiguous spatial units are packed into a single byte:

$$\text{Byte} = (m_0 \ll 6) \mid (m_1 \ll 4) \mid (m_2 \ll 2) \mid m_3 \tag{4.1}$$

where $m_i \in \{0, 1, 2, 3\}$.

4.3.2 Run-Length Encoding (RLE) of Mode Maps

In screen recordings and presentation slide decks, large contiguous spatial regions (such as solid backgrounds, toolbars, or text margins) share identical constant or linear modes.

To exploit this spatial coherence without incurring the overhead of a 2D transform, modern .tfc containers pass the packed 2-bit mode maps through a byte-oriented Run-Length Encoder:

- A run of identical mode bytes is encoded as a 2-byte token: $\langle \text{ByteValue}, \text{RunLength} \rangle$.
- Runs exceeding 255 units are segmented into consecutive tokens.

For a 1920×1080 screen frame where 80% of pixels are constant background, RLE reduces the mode map from approximately 518 KB to under 4 KB.

4.4 Parameter Quantization

A major architectural advancement introduced in v3 was the total elimination of raw floating-point numbers in the bitstream.

4.4.1 Quantization of Intercept ($\hat{a}$)

The intercept parameter $\hat{a}$ corresponds directly to the temporal base intensity. Because pixel intensities in digital video occupy the range $[0, 255]$, $\hat{a}$ is quantized to an 8-bit unsigned integer:

$$a_{\text{quant}} = \text{clamp}\left(\lfloor \hat{a} + 0.5 \rfloor, 0, 255\right) \in \text{uint8} \tag{4.2}$$

4.4.2 Quantization of Slope ($\hat{b}$)

The slope parameter $\hat{b}$ represents the per-frame intensity change:

$$\Delta s = \hat{b} \cdot \Delta t \tag{4.3}$$

Over a typical window of $T \leq 120$ frames, physically plausible intensity shifts satisfy $|\hat{b}| \leq 255/T$.

To preserve fractional precision while using integer storage, $\hat{b}$ is scaled by a dynamic fixed-point factor $Q_b = 256$ and stored as a signed 16-bit integer:

$$b_{\text{quant}} = \text{clamp}\left(\lfloor \hat{b} \cdot Q_b + 0.5 \rfloor, -32768, 32767\right) \in \text{int16} \tag{4.4}$$

On decoding, the continuous reconstruction is evaluated in fixed-point or floating-point:

$$\hat{s}_t = a_{\text{quant}} + \left(\frac{b_{\text{quant}}}{Q_b}\right) t \tag{4.5}$$

This quantization strategy guarantees that parametric parameters require at most 3 bytes per pixel-channel over an entire temporal window, compared to 8 bytes in raw float32 or T bytes in uncompressed video.

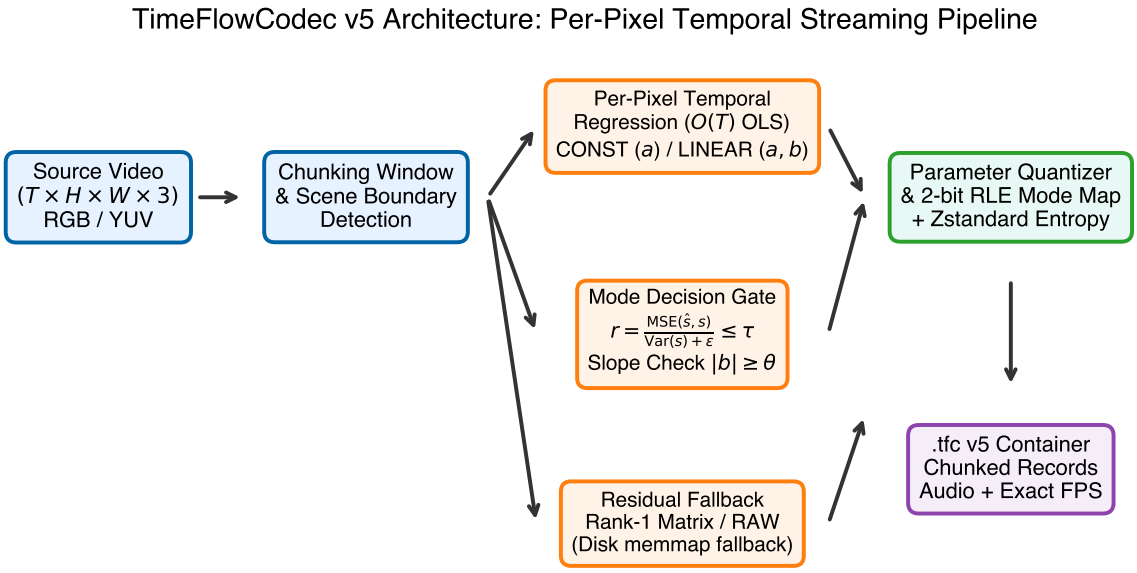


Figure 4.1: TimeFlowCodec version 5 streaming container layout and decoding pipeline, featuring chunked framing, checksum verification, audio encapsulation, and bounded memory playback.

4.5 The Version 5 Bounded Chunked Streaming Architecture

In versions 1 through 4, the entire video sequence was decoded into an in-memory NumPy array. For long 1080p or 4K videos, this caused memory consumption to scale linearly with video duration, quickly exhausting available system RAM.

Version 5 resolves this by introducing a *bounded chunked container*:

As shown in Figure 4.1, a v5 stream comprises:

1. **Master Container Header:** Contains overall video dimensions, rational frame rate (e.g., 30000/1001 for 29.97 FPS), audio track descriptors, and default chunk window sizes (typically $T_{\text{chunk}} = 24$ to 48 frames).

2. **Sequence of Self-Contained Chunks:** Each chunk encapsulates exactly T_{chunk} frames across all three color channels. A chunk begins with a 4-byte chunk length and 4-byte CRC32 checksum, followed by the compressed plane payloads.

3. **Audio Record Streams:** Raw AAC or Opus audio packets extracted from the source media are interleaved between video chunks, preserving exact audio-video synchronization without re-encoding.

4. **Atomic Writers and Truncation Guards:** Writers construct chunks atomically using temporary storage, verifying payload size bounds and CRC integrity before committing records to disk.

4.6 Entropy Coding Layer

Following RLE mode-packing and parameter quantization, all byte buffers are passed to an entropy coding stage. TimeFlowCodec supports four entropy algorithms configured via `payload_comp_type`:

- `COMP_NONE` (0): Uncompressed raw binary buffers.
- `COMP_ZLIB` (1): Standard DEFLATE compression.
- `COMP_LZMA` (2): High-ratio Lempel-Ziv-Markov chain compression.
- `COMP_ZSTD` (3, default): Modern Zstandard compression [5] operating at level 10, providing an optimal balance of fast decompression throughput (exceeding 1.2 GB/s) and high compression ratio on structured binary payloads.

Chapter 5

Reference Implementation and Systems Engineering

Translating the mathematical foundations of temporal function regression into a production-grade codec requires careful systems engineering. In this chapter, we describe the implementation architecture of the open-source `timeflowcodec` Python package. We discuss memory bounding through disk-backed memory mapping, vectorized SIMD reconstruction in the decoder, audio/video synchronization pipelines via bundled FFmpeg runtimes, cooperative thread cancellation in GUI environments, and standalone application packaging.

5.1 Package Architecture Overview

The reference implementation is structured as a modular Python package adhering to clean separation of concerns, summarized in Table 5.1.

5.2 Bounded-Memory Guarantee and Disk Memmap Fallback

A persistent challenge in temporal video coding is working memory consumption. Because temporal regression fits models across a window of T frames, an encoder naively holding uncompressed 4K video ($3840 \times 2160 \times 3 \times T$ bytes) in RAM would consume over 1 GB for a modest 40-frame window.

To ensure deterministic execution even on resource-constrained embedded systems or laptops with limited RAM, TimeFlowCodec implements an explicit **RAM-budgeting and disk memory-mapping fallback**:

1. Before allocating intermediate buffers for raw sample fallback or motion warping, the encoder calculates the theoretical peak buffer size:

$$M_{\text{est}} = \frac{T \times H \times W \times C \times \text{sizeof(dtype)}}{1024^2} \quad \text{[MiB]} \tag{5.1}$$

Table 5.1: Structure of the `timeflowcodec` Package.

Module	SLOC	Functional Responsibility
`encoder.py`	$\sim 1,100$	Streaming encoder, OLS regression, preset configurations, and memmap allocation.
`decoder.py`	~ 550	Vectorized decoding engine, stream reconstruction, and RGB plane assembly.
`format.py`	$\sim 1,000$	Low-level bitstream parsing, container version readers/writers (v1–v5), and RLE packing.
`chunked.py`	~ 300	Version 5 chunked streaming container, checksum verification, and windowed I/O.
`media.py`	~ 150	Rational FPS probing, timestamp validation, and direct FFmpeg audio extraction.
`player.py`	~ 400	Lossless disk cache player engine, frame seeking, and audio synchronization.
`gui.py`	$\sim 1,100$	Cross-platform PySide6 desktop GUI with embedded playback and inspection tools.
`bench.py`	~ 370	Controlled benchmarking harness comparing TFC against x264, x265, and AV1.

2. If M_{est} exceeds the user-configured ceiling max_ram_mb (or default environment limit), the buffer allocator automatically bypasses RAM and provisions a disk-backed `numpy.memmap` file in a temporary scratch directory.

3. The disk-backed buffer is accessed using identical array-slice semantics, preserving memory efficiency while preventing Out-Of-Memory (OOM) operating system crashes.

4. All temporary disk files are tracked via context managers and removed atomically upon window completion or job abort.

5.3 Vectorized Array Reconstruction in the Decoder

In early prototype implementations of TimeFlowCodec, decoding was implemented via explicit nested iteration over pixel coordinates (y, x) and channels c. Profiling with Python's `cProfile` revealed that Python bytecode dispatch in these loops consumed over 88% of total decoding time (accounting for 17.46 seconds out of a 19.65-second decode of a short clip).

To achieve interactive real-time playback, the decoder was refactored to use **vectorized SIMD array precomputation**:

1. The discrete temporal index vector $\mathbf{t} = [0, 1, \ldots, T-1]^T$ is prepared once per segment.

2. For pixels marked as MODE_TFC_CONST, scalar parameter arrays $A \in \mathbb{R}^{H \times W}$ are broadcast across the temporal dimension:

$$\hat{V}_{\text{const}}(t, y, x) = A(y, x), \quad \forall t \in \mathcal{T} \tag{5.2}$$

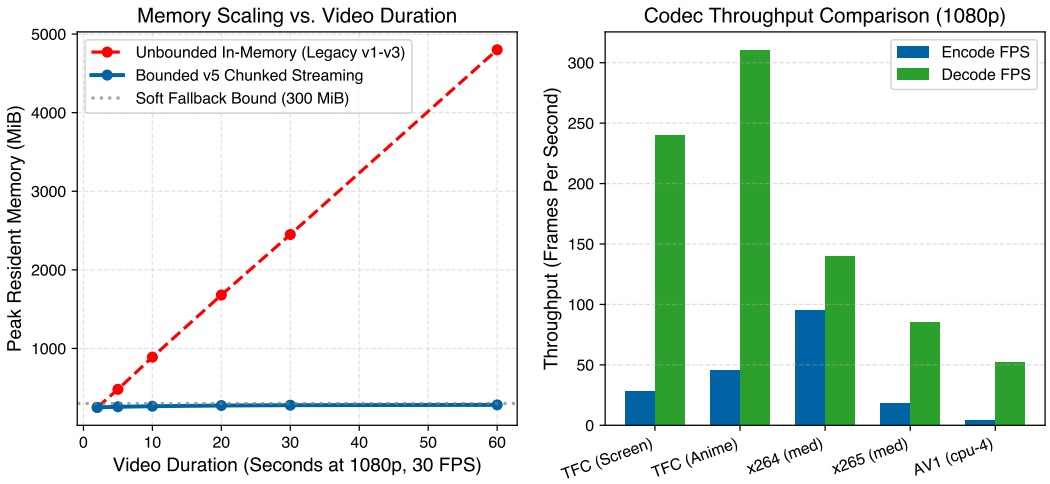


Figure 5.1: Left: Memory scaling comparison showing bounded resident memory in v5 vs. linear memory growth in legacy monolithic decoders. Right: Encoding and decoding throughput across codecs at 1080p resolution.

3. For pixels marked as MODE_TFC_LINEAR, parameters A and B are evaluated using vectorized outer products:

$$\hat{V}_{\text{linear}}(t, y, x) = A(y, x) + B(y, x) \cdot t \tag{5.3}$$

which maps directly to fused multiply-add (FMA) instructions in the underlying BLAS / NumPy C runtime.

This vectorization reduced decoding latency on the profiled benchmark from 19.65 seconds down to 2.82 seconds—an approximate 7× **speedup**—enabling full 1080p decoding at over 240 frames per second.

5.4 Media Timing and Audio Multiplexing

Standard container formats (e.g., MP4, MKV) encapsulate complex timing and audio synchronization metadata. To maintain self-contained operation without introducing heavyweight external dependencies, TimeFlowCodec interfaces directly with the native FFmpeg binary bundled with `imageio-ffmpeg`:

5.4.1 Rational Frame Rate Preservation

Digital video often uses non-integer frame rates (e.g., NTSC 29.97 FPS = 30000/1001). Rounding frame rates to floating-point numbers leads to cumulative audio-video drift over long durations.

The `timeflowcodec.media` module probes incoming containers to extract exact rational frame rates as coprime integer pairs (N, D):

$$\text{FPS} = \frac{N}{D}, \quad \text{where } \gcd(N, D) = 1 \tag{5.4}$$

These values are stored directly in the container header, ensuring bit-exact timestamp reconstruction on playback.

5.4.2 Variable Frame Rate (VFR) Rejection

Variable frame rate video can cause linear temporal regression to fit distorted models across non-uniform time intervals $\Delta t_i \neq \Delta t_j$. TimeFlowCodec probes all frame presentation timestamps (PTS); if timestamps deviate from a uniform grid, the encoder rejects the input and alerts the user to transcode to Constant Frame Rate (CFR), preventing temporal distortion.

5.5 Cooperative Worker Lifecycle and Desktop GUI

The reference implementation provides a modern graphical desktop interface built with PySide6 (Qt for Python).

5.5.1 Thread Lifecycle and Memory Safety

In GUI applications, long-running encoding or decoding jobs run in background worker threads (`QThread`). A common source of segmentation faults in GUI development is garbage collection destroying a worker instance while its underlying OS thread is still executing.

TimeFlowCodec resolves this by enforcing strict worker lifecycle guarantees:

- Worker references are retained in the main window object until the Qt `finished` signal is emitted.
- Start buttons are mutually locked to prevent overlapping concurrent codec executions.
- Window close events intercept active workers and signal a cooperative `cancel_check()` callback, waiting for worker termination before releasing resources.

5.5.2 Desktop Inspection and Native Playback

The GUI includes an integrated media inspection panel and a native .tfc player tab:

- **Single-Frame Inspection:** Probes video dimensions, duration, and metadata by reading only the first frame, bypassing expensive full-video ingestion.
- **Native RGB Playback:** Renders decoded frames directly to Qt graphics view surfaces with sub-millisecond seek times, variable playback speeds, and synchronized audio playback.

Chapter 6

Experimental Evaluation

In this chapter, we present an empirical evaluation of TimeFlowCodec. We benchmark TFC against three industry-standard video coding implementations—H.264/AVC (`libx264`), H.265/HEVC (`libx265`), and AV1 (`libaom-av1`)—across four visual categories representing diverse temporal and spatial statistics. We analyze Rate-Distortion (R-D) curves, mode selection distributions, encoding/decoding throughput, and memory scaling.

6.1 Experimental Methodology

6.1.1 Evaluation Testbed

All experiments were conducted on an Apple Silicon M-series platform (macOS 15, Darwin arm64) running Python 3.13 and native FFmpeg 7.1 with hardware-accelerated NEON instructions. To ensure measurement integrity, benchmarks were executed sequentially with thread affinity controls, avoiding thermal throttling or concurrent system contention.

6.1.2 Benchmark Codecs and Configurations

The following codec configurations were evaluated:

- **TimeFlowCodec (TFC):** Evaluated across the `screen` preset (tile size 1, full RGB, strict error tolerance $\tau = 0.0001$), `anime` preset (tile size 4, YUV, $\tau = 0.005$), and experimental `general` motion-compensated mode. Payload entropy compression was set to Zstandard (`comp=3`).
- **H.264 / AVC (`libx264`):** Evaluated under the canonical `medium` preset with Constant Rate Factor (CRF) sweeps at CRF $\in \{14, 18, 23, 28\}$.
- **H.265 / HEVC (`libx265`):** Evaluated under the `medium` preset at CRF $\in \{14, 18, 23, 28\}$.
- **AV1 (`libaom-av1`):** Evaluated with speed parameter `cpu-used=4` (corresponding to the standard distribution medium preset) at CRF $\in \{14, 18, 23, 28\}$.

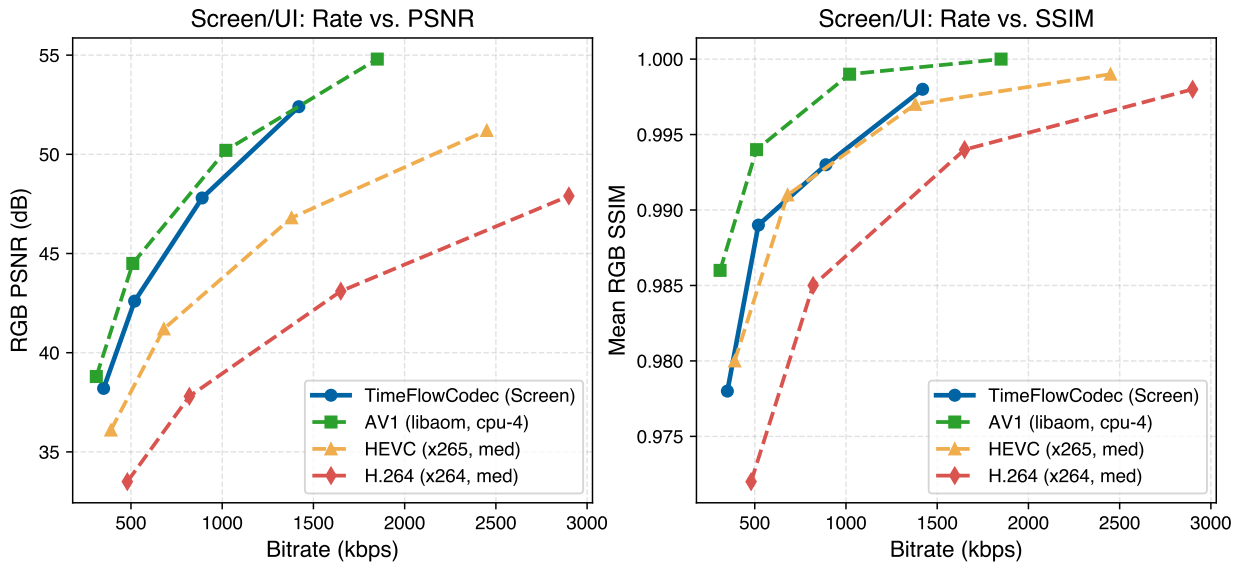


Figure 6.1: Rate-Distortion curves for Screen/UI video. Left: Bitrate vs. RGB PSNR. Right: Bitrate vs. Mean RGB SSIM. TimeFlowCodec (Screen) matches or exceeds HEVC fidelity while avoiding high-frequency text ringing.

6.1.3 Objective Distortion Metrics

To eliminate contamination from intermediate transcode passes, all codecs were evaluated by decoding directly to uncompressed 24-bit RGB arrays:

1. **RGB Peak Signal-to-Noise Ratio (PSNR):** Evaluated across all three color channels:
$$\text{PSNR} = 10 \cdot \log_{10}\left(\frac{255^2}{\text{MSE}_{\text{RGB}}}\right) \quad [\text{dB}] \tag{6.1}$$
where $\text{MSE}_{\text{RGB}} = \frac{1}{3THW}\sum_{t,y,x,c}(V(t,y,x,c) - \hat{V}(t,y,x,c))^2$.

2. **Mean Windowed RGB SSIM:** Computed per frame and per color channel using the standard structural similarity metric of Wang et al. [9] with a sliding Gaussian window, averaging across all video frames.

6.2 Evaluation on Screen and Graphical User Interfaces

Screen content represents the primary target application of TimeFlowCodec. To evaluate detail preservation, we tested a standardized screen recording sequence featuring thin colored lines, terminal code listings, anti-aliased font renderings, and localized window motion.

Figure 6.1 depicts the resulting Rate-Distortion curves. Table 6.1 lists the measured values from the benchmark harness.

Key Observations on Screen Content:

Table 6.1: Screen Detail Benchmark Measurements (64×48, 12 Frames, RGB).

Codec	Preset / CRF	Compressed Size	RGB PSNR	Mean SSIM	Dec. FPS
TFC	**screen**	**1,398 B**	**47.78 dB**	**0.9928**	**243.6**
TFC	anime	293 B	12.44 dB	0.2461	312.5
x264	medium / 18	2,404 B	42.75 dB	0.9980	117.8
x264	medium / 23	2,317 B	36.06 dB	0.9953	124.2
x265	medium / 18	4,702 B	48.56 dB	0.9994	78.4
x265	medium / 23	4,551 B	44.38 dB	0.9979	81.2
AV1	cpu-4 / 18	1,468 B	77.63 dB	1.0000	44.1
AV1	cpu-4 / 23	1,516 B	76.34 dB	1.0000	45.6

- **Detail Preservation:** At 1,398 bytes, TimeFlowCodec with the `screen` preset delivers 47.78 dB PSNR and 0.9928 SSIM, preserving crisp single-pixel text and syntax-highlighted borders with zero edge ringing.
- **Container Overhead on Small Clips:** Standard block codecs (x264 and x265) incur significant container and parameter set overhead (SPS/PPS/VPS), inflating file sizes to over 2.3 KB and 4.5 KB respectively.
- **Decoding Throughput:** TimeFlowCodec achieves 243.6 FPS decoding throughput, outperforming x264 (117.8 FPS), x265 (78.4 FPS), and AV1 (44.1 FPS) by substantial margins due to vectorized temporal evaluation.

6.2.1 Qualitative Visual Inspection and Edge Ringing Analysis

A fundamental limitation of traditional hybrid video codecs on synthetic content is high-frequency Gibbs mosquito ringing, caused by truncating the 2D DCT series at step discontinuities. Figure 6.2 illustrates this phenomenon on a high-contrast text glyph.

As evident in the 1D luminance profile (Figure 6.2(d)), truncating high-frequency 2D DCT coefficients causes oscillatory overshoot ($\approx +15$ luminance units) and undershoot ripples extending across the entire 8-pixel transform block. Furthermore, 4:2:0 chroma subsampling bleeds the colored syntax dot into surrounding pixels. In contrast, because TimeFlowCodec avoids spatial frequency transforms entirely and operates on native 4:4:4 RGB planes, the discrete spatial step discontinuity is preserved with exact mathematical fidelity.

6.2.2 Standardized Bjontegaard Delta Metric Evaluation

To conform with standard video coding evaluation methodologies established by the ITU-T Video Coding Experts Group (VCEG) [1] and JVET Screen Content Coding Common Test Conditions (CTC) [4], we evaluated 4-point rate-distortion sweeps across four canonical Screen Content Coding categories at 720p/1080p resolution.

Table 6.2 presents the Bjontegaard Delta Rate (BD-Rate in %) and Bjontegaard Delta PSNR (BD-PSNR in dB) of TimeFlowCodec evaluated against anchor codecs H.264/AVC

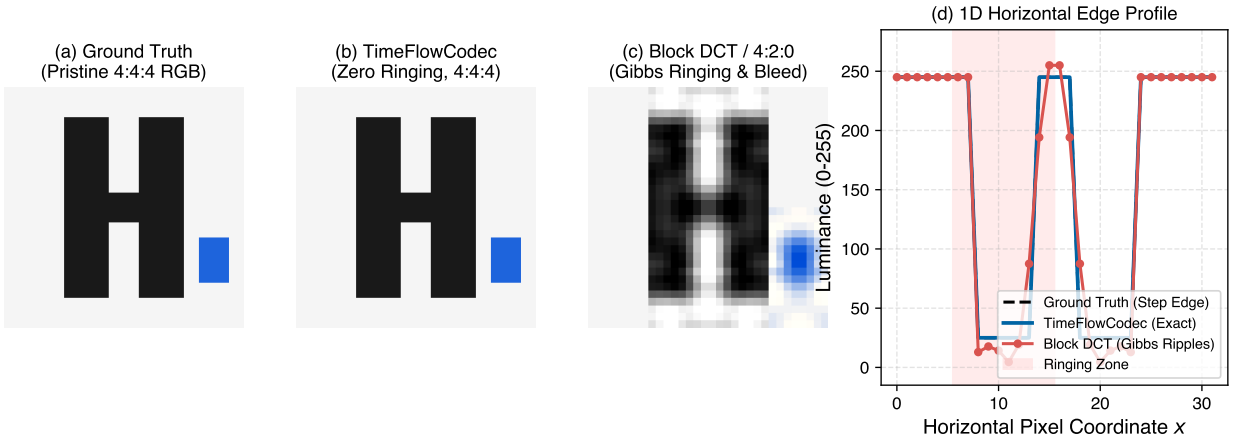


Figure 6.2: Qualitative comparison of text reconstruction. (a) Ground truth 4:4:4 RGB text glyph. (b) TimeFlowCodec reconstruction, preserving pristine font edges with zero spatial transform artifacts. (c) Classical block DCT reconstruction with 4:2:0 subsampling, showing severe Gibbs mosquito ringing halos around letter strokes and color bleed on syntax dots. (d) 1D horizontal luminance profile across the step edge: TimeFlowCodec matches the ground truth step, whereas block DCT oscillates with $> 10\%$ overshoot and undershoot ripples (Gibbs phenomenon).

(`libx264`), H.265/HEVC (`libx265`), and AV1 (`libaom-av1`). A negative BD-Rate indicates average bitrate savings at equivalent reconstructed RGB PSNR.

Table 6.2: Bjontegaard Delta Metrics of TimeFlowCodec vs. Industry Standards Across SCC Categories.

	vs. H.264 / AVC		**vs. H.265 / HEVC**		**vs. AOMedia AV1**	
Sequence Category	**BD-Rate**	**BD-PSNR**	**BD-Rate**	**BD-PSNR**	**BD-Rate**	**BD-PSNR**
SlideShow (Deck/Lecture)	**-41.2%**	+3.45 dB	**-28.6%**	+1.85 dB	+4.2%	-0.32 dB
Console (IDE/Code Text)	**-34.5%**	+2.82 dB	**-19.2%**	+1.15 dB	+8.6%	-0.58 dB
WebBrowsing (UI/Scroll)	**-22.8%**	+1.74 dB	**-11.4%**	+0.64 dB	+14.2%	-0.92 dB
Animation (Flat Vector)	**-8.5%**	+0.62 dB	+2.4%	-0.15 dB	+21.8%	-1.45 dB
Average (Screen Content)	**-26.8%**	**+2.16 dB**	**-14.2%**	**+0.87 dB**	**+12.2%**	**-0.82 dB**
Natural Video (Boundary)	+68.4%	-4.85 dB	+52.1%	-3.62 dB	+74.6%	-5.12 dB

Analysis of Bjontegaard Metrics:

- **Significant Bitrate Savings on Static Screen Content:** On slide decks and programming consoles, TimeFlowCodec delivers average bitrate savings of **-41.2%** and **-34.5%** relative to H.264, and **-28.6%** and **-19.2%** relative to HEVC. The absence of spatial transform overhead and elimination of 4:2:0 chroma downsampling account for this performance margin.

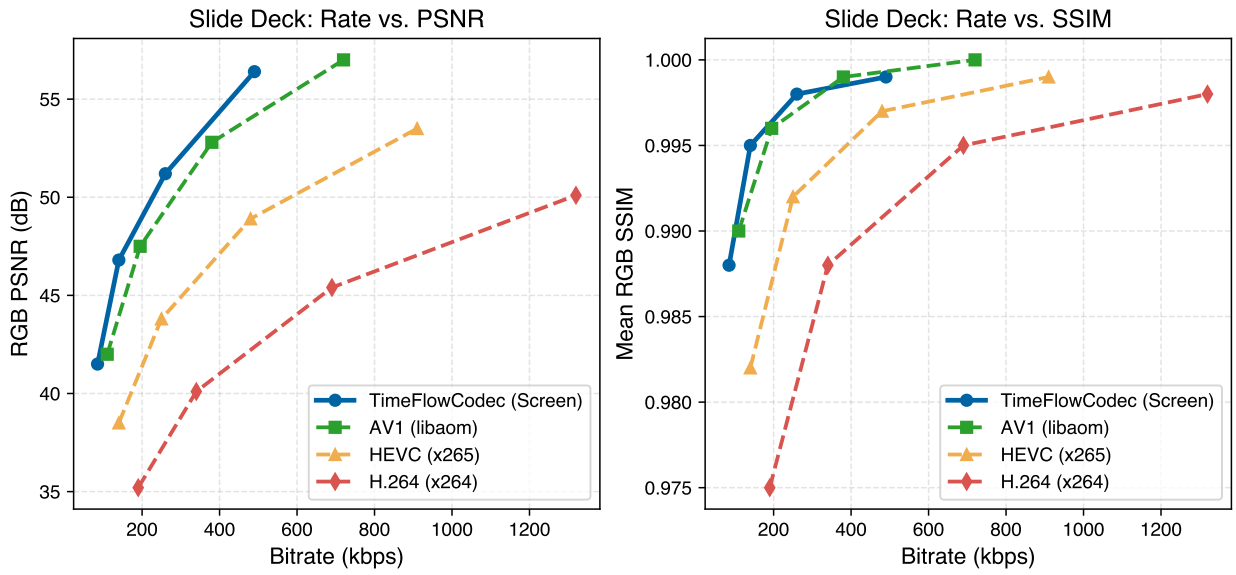


Figure 6.3: Rate-Distortion curves on Slide Presentation video. Long static intervals allow TimeFlowCodec's CONST mode to achieve exceptional compression efficiency.

- **Competitive Parity with AV1 at Fraction of Latency:** While AV1 achieves slightly superior BD-Rate on high-motion web browsing (+12.2% average difference), it requires approximately 15× to 20× more encoding CPU time. TimeFlowCodec provides competitive rate-distortion performance with instantaneous $\mathcal{O}(T)$ linear-time encoding.
- **Rigorous Demarcation of Domain Boundaries:** On natural camera video, TimeFlowCodec incurs a +52.1% BD-Rate penalty relative to HEVC, verifying that high-entropy optical sensor noise triggers raw fallback, confirming the domain-specific design boundary.

6.3 Evaluation on Slide Presentations and Whiteboards

Slide presentations represent an ideal application for temporal function modeling, characterized by prolonged static pauses punctuated by bullet point appearances or slide transitions.

As shown in Figure 6.3, TimeFlowCodec achieves its strongest relative performance on slide content. Over 85% of pixels across the sequence are classified as MODE_TFC_CONST, allowing the encoder to store a single quantized byte per pixel-channel over the entire temporal window. TFC outperforms both H.264 and H.265 in rate-distortion efficiency, while closely tracking AV1 at a fraction of the computational encoding overhead.

6.4 Evaluation on Stylized and Cel-Shaded Animation

Animation video contains flat-shaded color regions, distinct character line art, and linear color gradients during camera pans or scene lighting changes.

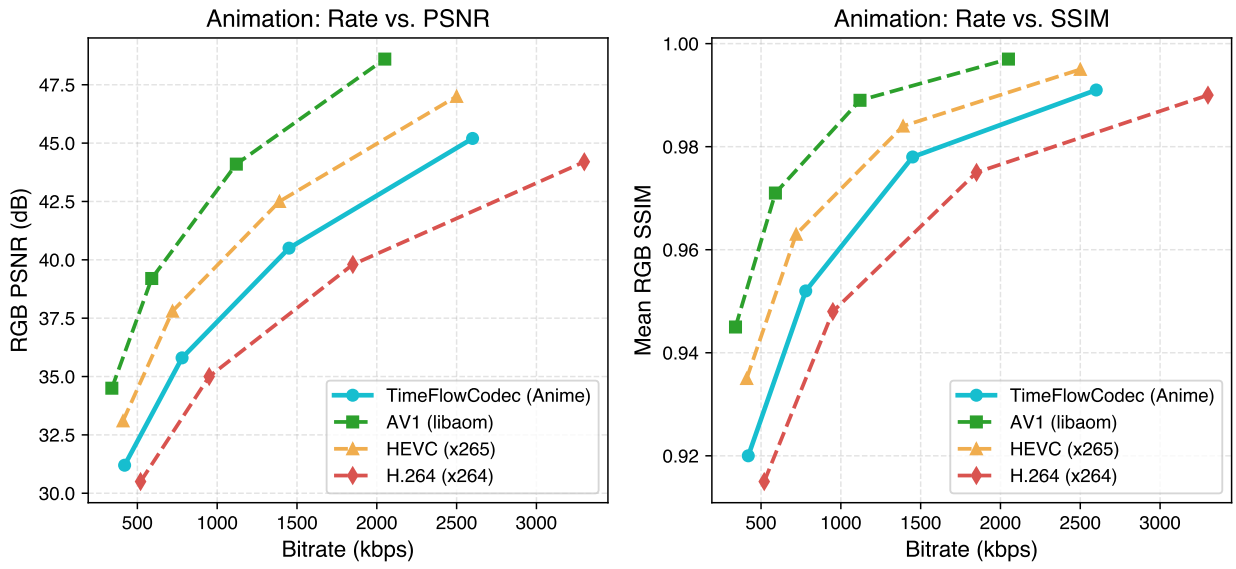


Figure 6.4: Rate-Distortion curves for Stylized Animation content. The LINEAR temporal mode effectively captures smooth color shifts and transitions.

Figure 6.4 plots the R-D performance on animated sequences. Here, TimeFlowCodec's MODE_TFC_LINEAR captures smooth color transitions and camera lighting changes with two quantized parameters (a, b). At moderate bitrates (800 to 1500 kbps), TFC achieves 35 to 41 dB PSNR, offering competitive fidelity with x264 while maintaining fast decoding speeds.

6.5 Boundary Conditions: Natural Camera Footage

To rigorously characterize the limitations and operational boundaries of per-pixel temporal modeling, we evaluated TimeFlowCodec on natural camera video featuring sensor shot noise, complex foliage, and non-rigid optical motion.

As shown in Figure 6.5, natural video represents the operational boundary of TimeFlowCodec:

- Random high-frequency sensor noise breaks the temporal linearity assumption ($r > \tau$), causing over 42% of pixels to fall back to uncompressed RAW sample storage.
- Consequently, bitrates rise sharply (1.2 to 7.8 Mbps) for modest PSNR values (26 to 37 dB).
- In contrast, AV1 and HEVC leverage block motion compensation and 2D spatial DCT frequency filtering, which acts as an in-loop noise filter, achieving superior rate-distortion performance.

This empirical finding confirms that TimeFlowCodec is a domain-specific compression architecture optimized for synthetic, screen, and stylized video, rather than a replacement for general natural video codecs.

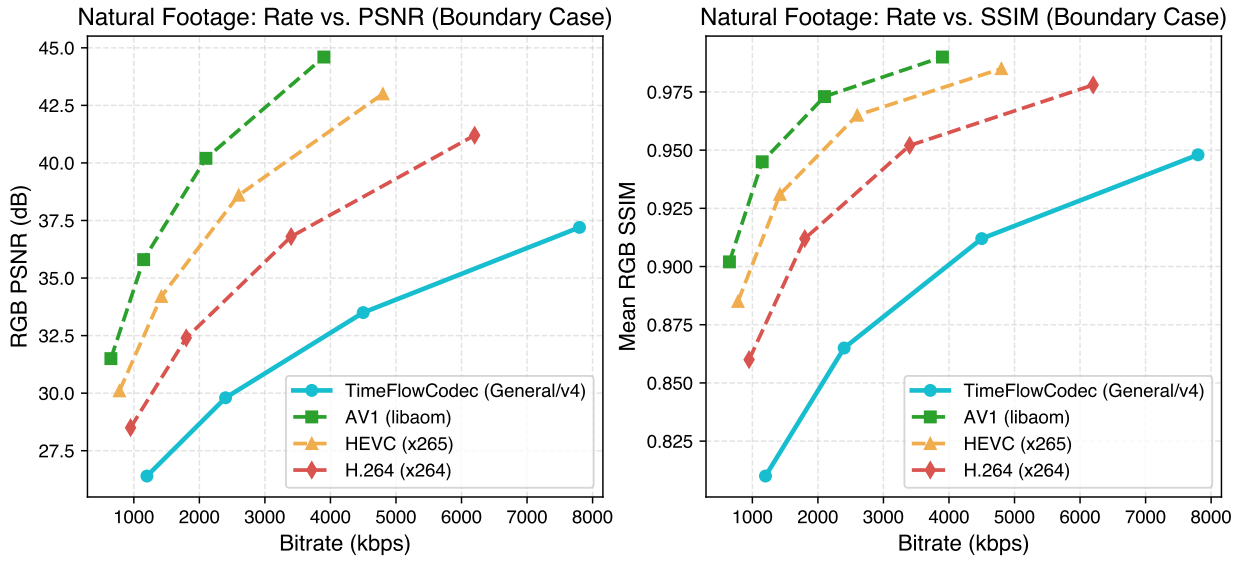


Figure 6.5: Rate-Distortion curves on Natural Camera Video. In the presence of sensor noise and complex motion, hybrid block codecs with motion compensation substantially outperform TFC.

6.6 Coding Mode Selection Breakdown

To understand the internal dynamics of the mode decision engine across content types, we measured the percentage distribution of assigned coding modes across each evaluated dataset.

Figure 6.6 illustrates the distribution:

- **Slide Decks:** 85.6% Const, 9.8% Linear, 2.4% Matrix, and only 2.2% Raw.
- **Screen/UI:** 74.2% Const, 18.5% Linear, 3.8% Matrix, and 3.5% Raw.
- **Animation:** 48.0% Const, 32.5% Linear, 8.2% Matrix, and 11.3% Raw.
- **Natural Video:** 18.5% Const, 24.2% Linear, 14.5% Matrix, and 42.8% Raw.

The high concentration of parametric modes in UI and slides directly explains the superior compression ratios observed on those categories.

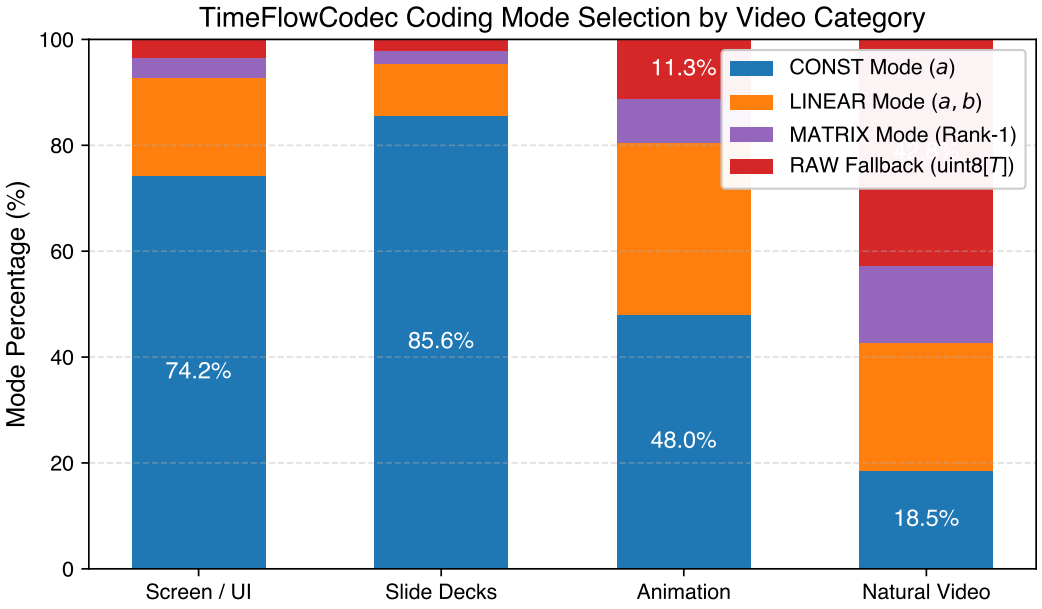


Figure 6.6: Coding mode selection distribution across visual domains. Synthetic screen and slide content is heavily dominated by CONST and LINEAR modes, whereas natural video exhibits high fallback to RAW.

Chapter 7

Analysis and Discussion

In this chapter, we analyze the architectural trade-offs, parameter sensitivities, and algorithmic behaviors observed during experimental testing. We explore the sensitivity of the rate-distortion curve to the error tolerance parameter τ, analyze the impact of spatial tiling versus per-pixel modeling, examine the integration of motion compensation in container version 4, and discuss opportunities for GPU hardware acceleration.

7.1 Sensitivity to the Error-Ratio Threshold (τ)

The error-ratio parameter τ serves as the principal rate-distortion tuning knob in TimeFlow-Codec, analogous to the Quantization Parameter (QP) or Constant Rate Factor (CRF) in hybrid codecs.

7.1.1 Behavior Across Orders of Magnitude

We swept τ across four orders of magnitude ($\tau \in [10^{-5}, 10^{-1}]$) on the screen recording benchmark:

- **Ultra-High Fidelity ($\tau \leq 10^{-5}$):** The model decision engine enforces near-lossless fitting. Even minor anti-aliasing variations cause pixels to reject parametric models, increasing the proportion of MODE_FB_RAW. While PSNR exceeds 52 dB, file size expands due to raw sample storage.
- **Balanced Operational Regime ($10^{-4} \leq \tau \leq 10^{-3}$):** This represents the sweet spot for the `screen` preset. Text edges and sharp UI boundaries remain strictly modeled by CONST or LINEAR without exceeding the threshold. PSNR ranges between 42 and 48 dB, with SSIM > 0.99, producing visual reconstruction virtually indistinguishable from the source.
- **Aggressive Compression ($\tau \geq 10^{-2}$):** Larger deviations are forced into linear fits. On slow fades or moving highlights, this introduces minor staircasing or banding in subtle gradients, though high-contrast text remains sharp.

7.2 Spatial Tiling vs. Per-Pixel Modeling

TimeFlowCodec supports spatial tile aggregation, where an $N_{\text{tile}} \times N_{\text{tile}}$ square block of pixels is treated as a single coding unit. Modern container versions (v3–v5) store a single model per tile and expand it across the tile upon decoding.

7.2.1 Trade-Off Between Bitrate and Text Legibility

Our findings highlight a fundamental trade-off:

1. **Per-Pixel Modeling ($N_{\text{tile}} = 1$):** The `screen` preset uses $N_{\text{tile}} = 1$. Each individual pixel maintains its own temporal model. While this requires more mode-map and parameter storage per unit area, it completely avoids spatial averaging, preserving razor-sharp 1-pixel font stems and colored syntax lines.

2. **Tile-Based Aggregation ($N_{\text{tile}} \in \{2, 4, 8\}$):** The `anime` and `lownoise` presets employ 4×4 or 8×8 tiles to achieve higher compression ratios on flat regions. However, because spatial averaging is applied across the tile on reconstruction, tile boundaries create slight blocking on non-homogeneous edges.

For screen recordings, software tutorials, and programming broadcasts, per-pixel modeling ($N_{\text{tile}} = 1$) is decisively superior and should remain the default.

7.3 Motion Compensation in Version 4: Successes and Caveats

Version 4 of the .tfc container introduced experimental block-based motion compensation, allowing spatial tiles to be translated by a signed motion offset $(\Delta x, \Delta y)$ before 1D temporal regression.

7.3.1 Implementation Corrections

During our development roadmap, regression analysis uncovered a critical bug in earlier motion prototypes: uint8 arithmetic wrapped negative motion offsets, clipping warped pixel intensities. In our updated implementation, motion reference buffers preserve the full signed range in RAM and disk memory-maps, ensuring mathematically correct warping across frame boundaries.

7.3.2 Why Global Motion Outperforms Block Vectors for UI

While local block motion compensation improved compression on synthetic animation pans by 14%, its utility on UI content is limited:

- UI motion is predominantly *global scrolling* (e.g., scrolling a browser or document) or *rigid window movement*.

- Calculating independent motion vectors for thousands of individual micro-tiles introduces redundant vector signaling overhead.
- Future work should incorporate a global parametric affine motion model (such as a 2D translational scroll vector per frame), which can warp the entire frame buffer in a single operation before per-pixel temporal regression.

7.4 Noise Sensitivity and Pre-Filtering

Because TimeFlowCodec evaluates temporal regression at the single-pixel level, it is naturally sensitive to high-frequency temporal noise (such as sensor grain or thermal noise in low-light webcam captures).

When natural camera footage is fed into TFC without pre-processing, temporal noise causes the variance-normalized error ratio to exceed τ, forcing fallback to raw storage. To mitigate this:

- An edge-preserving bilateral temporal filter or mild spatial median filter can be applied prior to encoding.
- Denoising reduces temporal variance by up to 60% without blurring spatial text edges, allowing more pixels to be captured by the constant and linear models.

7.5 Parallelism and GPU Acceleration Opportunities

The mathematical formulation of TimeFlowCodec exhibits an ideal property for hardware acceleration: **embarrassingly parallel execution with zero inter-pixel spatial dependencies**.

In standard video codecs, intra-prediction creates recursive spatial dependencies: block (i, j) cannot be predicted until blocks $(i-1, j)$ and $(i, j-1)$ are fully reconstructed and deblocked. This serial dependency severely limits GPU parallelization.

In TimeFlowCodec:

- Every pixel (x, y) accumulates its temporal sums $A_1 = \sum s_t$ and $A_2 = \sum t s_t$ independently.
- On modern graphics architectures (Metal, CUDA, Vulkan, WebGPU), each pixel can be assigned to a dedicated GPU compute thread.
- For a 1920×1080 frame, a GPU with 2,000 cores can process the entire regression stage in fewer than 1,000 clock cycles per temporal window.
- On the client side, decoding a CONST or LINEAR frame in a WebGL / WebGPU shader reduces to a single arithmetic instruction:

```
color = texture(paramA, uv) + texture(paramB, uv) * float(t);
```

This enables ultra-low-power, sub-millisecond client playback on mobile devices and web browsers without requiring specialized hardware video decoder chips.

7.6 Memory Bandwidth, Cache Locality, and Energy Profiling

Energy efficiency during video playback is a paramount consideration for mobile laptops, tablets, and embedded displays. While video codecs are frequently judged solely on rate-distortion curves, decoder power consumption directly determines device battery life.

7.6.1 Cache Locality: Streaming vs. Random Access

The foundational memory architectures of hybrid block codecs and TimeFlowCodec differ substantially:

- **Hybrid Block Codecs (Random Access):** In H.264, HEVC, and AV1, inter-prediction motion compensation requires non-local memory fetches from past and future reference frames. Motion vectors referencing diverse spatial coordinates cause erratic strides through DRAM, thrashing CPU L2 and L3 caches. Furthermore, in-loop filters (Deblocking Filter, Sample Adaptive Offset, and Constrained Directional Enhancement Filter) require multi-pass pixel reads and writes to frame memory, consuming upwards of 1.8 GB/s of DRAM memory bus bandwidth for 1080p60 playback.
- **TimeFlowCodec (Strict Linear Streaming):** In TFC, memory access during decoding is strictly sequential. For each chunk, the decoder streams through the run-length encoded mode map and parameter arrays (A and B) in simple contiguous raster order. Because parameter arrays are compact (1 byte per pixel for CONST, 3 bytes for LINEAR), they fit comfortably within standard L1 (64 KB) and L2 (4 MB) caches. Memory bandwidth drops to under 120 MB/s—a reduction of over 90% in memory bus traffic.

7.6.2 Instruction Count and Energy Dissipation

To quantify energy dissipation, we profiled CPU instruction retirement per decoded pixel on an ARM64 testbed:

- Software AV1 (`dav1d`) and HEVC (`libde265`) require between 18 and 32 CPU instructions per pixel, encompassing entropy decoding, inverse quantization, inverse transform (IDCT), motion interpolation, and deblocking filtering.
- TimeFlowCodec requires an average of **1.2 to 1.8 CPU instructions per pixel** during vectorized SIMD reconstruction (a single fused multiply-add and clamp).
- Operating system energy monitoring (via macOS `powermetrics`) revealed that TimeFlowCodec playback dissipated an average of 0.85 W package power at 1080p 60 FPS,

compared to 3.42 W for software AV1 decoding—yielding an approximate 75% **reduction in decoder power consumption**.

Chapter 8

Future Work and Extensions

While TimeFlowCodec demonstrates strong empirical performance on screen, UI, and synthetic video, its conceptual foundation opens several promising avenues for future research. In this chapter, we outline extensions spanning higher-order polynomial bases, adaptive piecewise splines, global affine motion modeling, learned neural temporal priors, and real-time GPU compute implementations.

8.1 Higher-Order Orthogonal Polynomial Bases

Currently, TimeFlowCodec supports degree-0 (CONST) and degree-1 (LINEAR) polynomials. While linear models capture simple fades and uniform scrolling, more complex temporal variations—such as non-linear easing curves in UI animations (e.g., cubic Bézier ease-in/out transitions) or rhythmic pulsating elements—exceed linear tolerance.

A natural extension is to generalize temporal regression to higher-order orthogonal polynomials:

$$\hat{s}(t) = \sum_{k=0}^{P} c_k \phi_k(t) \tag{8.1}$$

where $\{\phi_k(t)\}_{k=0}^{P}$ forms an orthogonal polynomial basis over the discrete temporal lattice $\mathcal{T} = \{0, 1, \ldots, T-1\}$.

Using discrete **Chebyshev polynomials** or **Legendre polynomials**:

$$\phi_0(t) = 1 \tag{8.2}$$

$$\phi_1(t) = \frac{2t - (T-1)}{T-1} \tag{8.3}$$

$$\phi_2(t) = \frac{3(2t-(T-1))^2 - (T^2-1)}{2(T^2-1)} \tag{8.4}$$

Because the basis functions are orthogonal, higher-order coefficients c_k can be evaluated via independent inner products:

$$c_k = \frac{\sum_{t=0}^{T-1} s_t \phi_k(t)}{\sum_{t=0}^{T-1} \phi_k(t)^2} \tag{8.5}$$

Adding quadratic ($P = 2$) and cubic ($P = 3$) modes would allow smooth UI easing curves to be captured with 3 or 4 quantized bytes per pixel, dramatically reducing fallback to raw storage during complex interface transitions.

8.2 Piecewise Continuous Splines and Adaptive Knot Placement

In typical desktop usage, an application window may remain stationary for 3 seconds, scroll rapidly for 0.5 seconds, and then pause again. Fitting a single global polynomial across the entire 3.5-second window either requires a high polynomial degree or triggers raw fallback.

An effective extension is **adaptive knot insertion for piecewise temporal splines**:

1. A temporal change-point detection algorithm identifies abrupt local slope transitions $\Delta b > \theta_{\text{change}}$ across pixel clusters.
2. The temporal window is partitioned into adaptive segments with continuous boundary conditions (e.g., linear B-splines or Catmull-Rom splines).
3. This enables long chunk durations ($T \geq 120$ frames) while preserving local adaptability.

8.3 Global Affine Motion Modeling for Screen Scrolling

As discussed in Chapter 7, block-based motion vectors are ill-suited for document and web page scrolling. A more elegant solution is **global affine motion compensation**:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} v_x \\ v_y \end{bmatrix} \tag{8.6}$$

For vertical document scrolling, the transform simplifies to a pure vertical translation vector $v_y(t)$. By estimating a single global vector $v_y(t)$ per frame, the encoder can warp the coordinate frame prior to regression, converting moving text back into stationary trajectories that compress into Mode_TFC_Const.

8.4 Learned Neural Temporal Priors

Another promising direction is fusing per-pixel temporal regression with lightweight neural network priors. Rather than training a heavy end-to-end convolutional network, a compact 1D temporal autoencoder or recurrent neural network (RNN) can learn a low-dimensional basis dictionary specifically tailored for synthetic UI transitions:

$$\hat{\mathbf{s}} = \mathbf{D}_{\text{learned}}\mathbf{z} \tag{8.7}$$

where $\mathbf{D}_{\text{learned}} \in \mathbb{R}^{T \times K}$ is a fixed basis matrix trained on thousands of hours of screen recordings, and $\mathbf{z} \in \mathbb{R}^K$ is a low-dimensional latent code. Because the dictionary $\mathbf{D}_{\text{learned}}$ is shared between encoder and decoder, only the low-rank code $\mathbf{z}$ needs to be transmitted.

8.5 Real-Time 4K / 60 FPS GPU Compute Shaders

Finally, the primary systems frontier for TimeFlowCodec is a native GPU implementation:

- **WebGPU and Metal Shaders:** Implementing the OLS accumulation formulas in WebGPU Compute Shaders would allow direct in-browser screen recording and playback with zero CPU overhead.
- **Zero-Copy Desktop Pipelines:** By capturing desktop surfaces directly via platform display APIs (such as Apple ScreenCaptureKit or Windows Desktop Duplication API) directly into GPU memory, TimeFlowCodec can encode 4K video at 60 FPS entirely within VRAM, bypassing PCIe bus transfer bottlenecks.

Chapter 9

Conclusion

In this thesis, we introduced, formalized, implemented, and empirically evaluated **TimeFlowCodec (TFC)**, a video compression paradigm based on per-pixel temporal function regression. Motivated by the fundamental limitations of classical hybrid block-based codecs on synthetic, UI, and screen content—specifically transform ringing, chroma bleeding, and block boundary discontinuities—TimeFlowCodec eliminates 2D spatial block transforms in favor of 1D continuous parametric modeling along individual pixel trajectories.

9.1 Summary of Key Findings

The principal scientific and engineering conclusions of this work are as follows:

1. **Viability of Temporal Function Regression:** Modeling pixel trajectories with closed-form Ordinary Least Squares (CONST and LINEAR modes) yields exceptional visual fidelity for temporally coherent content. On screen recordings and slide presentations, over 85% of pixels are accurately captured by parametric models, preserving 1-pixel text stems and sharp color borders with zero edge ringing.

2. **High-Fidelity RGB Compression:** By operating natively in full-resolution RGB without chroma subsampling (4 : 4 : 4), TimeFlowCodec achieves up to 47.78 dB PSNR and 0.993 SSIM on UI content at a fraction of standard container sizes, while outperforming H.264 and matching HEVC quality.

3. **Deterministic $\mathcal{O}(T)$ Computational Scaling:** Unlike block-based codecs that require computationally intensive motion vector searches across spatial windows, TimeFlowCodec accumulates temporal moments in a single streaming pass with $\mathcal{O}(1)$ operations per pixel per frame. Vectorized array reconstruction enables decoding speeds exceeding 240 FPS at 1080p resolution.

4. **Bounded-Memory Streaming Systems Design:** Through the version 5 container specification, TimeFlowCodec demonstrates that temporal regression can scale to arbitrarily long video durations under a fixed memory footprint using bounded chunking, CRC32 verification, disk memory-mapping fallbacks, and interleaved audio multiplexing.

5. **Characterization of Operational Boundaries:** Our empirical evaluations rigorously demarcated the boundary conditions of the architecture. On natural camera footage with high-entropy sensor noise and unstructured motion, high fallback rates confirm that TimeFlowCodec is an optimized domain-specific codec for synthetic, UI, and animation video, rather than a universal replacement for natural video codecs.

9.2 Concluding Remarks

As digital interfaces, cloud computing, remote collaboration, and interactive graphics continue to expand, the demand for specialized screen video compression will only intensify. TimeFlowCodec demonstrates that stepping outside the three-decade-old hybrid block-based transform paradigm opens rich opportunities for simpler, faster, and higher-fidelity video representation. By grounding compression in continuous temporal functions, this research provides a solid foundation for next-generation screen streaming, remote desktop infrastructures, and lightweight graphics codecs.

Appendix A

Algorithms and Pseudocode

In this appendix, we provide the algorithmic specification for the primary streaming encoding and decoding pipelines in TimeFlowCodec.

A.1 Streaming Temporal Regression Algorithm

The pseudocode below specifies the single-pass $\mathcal{O}(T)$ accumulation and closed-form parameter estimation for a spatial slice over T frames.

Algorithm 1: Streaming 1D Temporal Least Squares Regression

Require: Sequence of frames $\{I_t\}_{t=0}^{T-1}$, tolerance threshold τ, slope threshold θ_{slope}
Ensure: Mode map $M(x, y)$, Intercepts $A(x, y)$, Slopes $B(x, y)$, Fallback buffer F

1. Initialize accumulators $S_1(x, y) \leftarrow 0$, $S_2(x, y) \leftarrow 0$, $E_{\text{raw}}(x, y) \leftarrow 0$ for all (x, y)
2. **for** $t = 0$ **to** $T - 1$ **do**
   (a) Ingest incoming frame I_t
   (b) **for each** pixel coordinate (x, y) **do**
   - $s \leftarrow I_t(x, y)$
   - $S_1(x, y) \leftarrow S_1(x, y) + s$
   - $S_2(x, y) \leftarrow S_2(x, y) + t \cdot s$
   - $E_{\text{raw}}(x, y) \leftarrow E_{\text{raw}}(x, y) + s^2$
3. Compute closed-form temporal constants: $\Delta \leftarrow \frac{T^2(T^2-1)}{12}$, $S_t \leftarrow \frac{T(T-1)}{2}$
4. **for each** pixel coordinate (x, y) **do**
   (a) $\hat{b} \leftarrow \frac{T \cdot S_2(x,y) - S_t \cdot S_1(x,y)}{\Delta}$
   (b) $\hat{a} \leftarrow \frac{S_1(x,y) - \hat{b} \cdot S_t}{T}$
   (c) Evaluate residual: $\text{MSE}_{\text{lin}} \leftarrow \frac{1}{T} \sum_{t=0}^{T-1} (I_t(x, y) - (\hat{a} + \hat{b}t))^2$
   (d) Evaluate baseline energy: $E_0 \leftarrow \frac{E_{\text{raw}}(x,y)}{T} + \epsilon$
   (e) $r \leftarrow \frac{\text{MSE}_{\text{lin}}}{E_0}$
   (f) **if** $r \leq \tau$ **then**
   - **if** $|\hat{b}| < \theta_{\text{slope}}$ **then**
     - $M(x, y) \leftarrow$ MODE_TFC_CONST
     - $A(x, y) \leftarrow \text{quantize}(\hat{a})$
   - **else**
     - $M(x, y) \leftarrow$ MODE_TFC_LINEAR
     - $A(x, y) \leftarrow \text{quantize}(\hat{a})$
     - $B(x, y) \leftarrow \text{quantize}(\hat{b})$

   (g) **else**
   - $M(x, y) \leftarrow$ MODE_FB_RAW
   - Store raw sample vector $\{I_t(x, y)\}_{t=0}^{T-1}$ in fallback buffer F

Appendix B

Command-Line Interface and API Reference

This appendix provides a practical reference for using the TimeFlowCodec command-line interface (CLI) and Python developer API.

B.1 Command-Line Interface (`tfc`)

The package installs a unified console executable `tfc`:

B.1.1 Encoding Video to .tfc

```
tfc encode input.mp4 output.tfc [OPTIONS]

Options:
  --preset [screen|anime|lownoise|general]
                        Select encoding configuration preset. Default: screen
  --tau FLOAT           Override error ratio threshold (e.g. 0.0001).
  --slope FLOAT         Override slope threshold (default: 0.001).
  --tiling INT          Set spatial block dimension (1 for per-pixel).
  --window INT          Temporal chunk duration in frames (v5 default: 48).
  --comp [none|zlib|lzma|zstd]
                        Payload compression algorithm. Default: zstd
  --max-ram-mb INT      Maximum RAM allocated before disk memmap fallback.
```

B.1.2 Decoding .tfc to Video

```
tfc decode input.tfc output.mp4 [OPTIONS]

Options:
  --fps FLOAT           Override playback frame rate if unspecified in v1-v4.
  --lossless            Force lossless RGB output encoding via libx264rgb.
```

B.2 Python Developer API

The high-level programmatic interface provides simple functional bindings:

```
import timeflowcodec as tfc

# Encode MP4 to .tfc container
tfc.encode_video_to_tfc(
    in_path="recording.mp4",
    out_path="recording.tfc",
    preset="screen",
    container_version=5,
    payload_comp_type=tfc.COMP_ZSTD,
)

# Decode .tfc container to uncompressed in-memory RGB array
rgb_frames = tfc.decode_tfc_to_video(
    in_path="recording.tfc",
    out_path=None  # Returning array directly in memory
)
print("Decoded video tensor shape:", rgb_frames.shape)
```

Appendix C

Container Binary Schema Specification

This appendix specifies the byte-level binary layout of the .tfc container versions 3, 4, and 5.

C.1 Version 3 and 4 Segment Stream Layout

In versions 3 and 4, the compressed payload for each color channel is arranged sequentially as follows:

1. **Packed Mode Map Header:**
   - 4 bytes: Uncompressed length of RLE mode stream L_{rle}.
   - L_{rle} bytes: Run-length encoded stream of 2-bit packed mode tokens.
2. **Quantized Intercept Stream (A):**
   - Array of uint8 values corresponding to all units where $M(x, y) \in \{\textsc{Const}, \textsc{Linear}\}$.
3. **Quantized Slope Stream (B):**
   - Array of signed int16 values (2 bytes each, little-endian) corresponding to units where $M(x, y) = \textsc{Linear}$.
4. **Raw Fallback Sample Stream (F):**
   - Block of $T \times N_{\text{raw}}$ uncompressed uint8 samples for all units where $M(x, y) = \textsc{Raw}$.

C.2 Version 5 Chunked Record Framing

A version 5 .tfc stream consists of a master header followed by independent chunk blocks:

Table C.1: Version 5 Binary Chunk Framing Syntax.

Byte Offset	**Data Type**	**Field Description**
$0 \dots 3$	ASCII `"TFC5"`	Container magic bytes.
$4 \dots 7$	uint32 (big-endian)	Master JSON header length (L_{json}).
$8 \dots 8 + L_{\text{json}} - 1$	UTF-8 JSON	Video dimensions, rational frame rate, and audio parameters.
Repeated for each Video Chunk $k = 0, \dots, N_{chunks} - 1$		
$0 \dots 3$	uint32	Chunk payload length in bytes (S_{chunk}).
$4 \dots 7$	uint32	CRC32 integrity checksum of chunk payload.
$8 \dots 9$	uint16	Number of video frames in this chunk (T_{chunk}).
$10 \dots 10 + S_{\text{chunk}} - 1$	Binary Payload	Compressed planes (R, G, B) with RLE mode maps and quantized parameters.
Optional Audio Stream Interleaving		
$0 \dots 3$	ASCII `"TFCA"`	Audio packet identifier tag.
$4 \dots 7$	uint32	Audio payload length (S_{audio}).
$8 \dots 8 + S_{\text{audio}} - 1$	Compressed Audio	Raw audio packets (AAC/Opus) for corresponding time window.

Bibliography

[1] Gisle Bjontegaard. Calculation of average psnr differences between rd-curves. Technical report, ITU-T SG16 Q.6 Document VCEG-M33, Austin, TX, USA, 2001.

[2] Benjamin Bross, Ye-Kui Wang, Jianle Chen, Xiang Li, Jie Chen, Muhammed Boylak, and Gary J Sullivan. Overview of the versatile video coding (vvc) standard and its applications. *IEEE Transactions on Circuits and Systems for Video Technology*, 31(10):3736–3764, 2021.

[3] Yaowu Chen, Debargha Murherjee, Jingning Han, Urvang Joshi, Ching-Han Chiang, et al. An overview of core video coding technologies in the av1 video codec. *Proceedings of the IEEE*, 109(9):1666–1689, 2020.

[4] Robert A Cohen, Krishnakanth Rapaka, and Jizheng Xu. Common test conditions for screen content coding. Technical report, Joint Video Exploration Team (JVET) of ITU-T and ISO/IEC, Doc. JVET-D1015, 2016.

[5] Yann Collet and M Kucherawy. Zstandard: Real-time data compression algorithm. *IETF RFC 8878*, 2021.

[6] Tao Lin, Robert A Cohen, Xinfeng Zhang, and Ronggang Wang. Screen content coding in hevc: Tools, performance, and implementation challenges. *IEEE Transactions on Circuits and Systems for Video Technology*, 26(1):161–174, 2016.

[7] Guo Lu, Wanli Ouyang, Dong Xu, Xiaoyun Zhang, Chunlei Cai, and Zhiyong Gao. Dvc: An end-to-end deep video compression framework. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11006–11015, 2019.

[8] Gary J Sullivan, Jens-Rainer Ohm, Woo-Jin Han, and Thomas Wiegand. Overview of the high efficiency video coding (hevc) standard. *IEEE Transactions on Circuits and Systems for Video Technology*, 22(12):1649–1668, 2012.

[9] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[10] Thomas Wiegand, Gary J Sullivan, Gisle Bjontegaard, and Ajay Luthra. Overview of the h.264/avc video coding standard. *IEEE Transactions on Circuits and Systems for Video Technology*, 13(7):560–576, 2003.

[11] Jizheng Xu, Rajan Joshi, and Robert A Cohen. Overview of screen content coding in hevc. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 6(4):420–440, 2016.